

|           |                                                   |           |
|-----------|---------------------------------------------------|-----------|
| <b>1</b>  | <b>Reminder</b>                                   | <b>1</b>  |
| 1.1       | Observations and Tricks                           | 1         |
| 1.2       | Bug List                                          | 1         |
| <b>2</b>  | <b>Init (Linux)</b>                               | <b>1</b>  |
| 2.1       | vimrc                                             | 1         |
| 2.2       | bashrc                                            | 1         |
| <b>3</b>  | <b>Basic</b>                                      | <b>1</b>  |
| 3.1       | Template (Using Codebook)                         | 1         |
| 3.2       | PBDS, Random                                      | 1         |
| 3.3       | Debug                                             | 2         |
| 3.4       | SVG Writer                                        | 2         |
| 3.5       | Python                                            | 2         |
| 3.6       | Stress Tests                                      | 2         |
| <b>4</b>  | <b>Data Structure</b>                             | <b>2</b>  |
| 4.1       | Mo's Algorithm                                    | 2         |
| 4.2       | CDQ                                               | 2         |
| 4.3       | Persistent Treap                                  | 3         |
| 4.4       | Li Chao Tree                                      | 3         |
| 4.5       | Time Segment Tree                                 | 3         |
| <b>5</b>  | <b>DP</b>                                         | <b>4</b>  |
| 5.1       | SOS DP                                            | 4         |
| 5.2       | Divide and Conquer DP                             | 4         |
| 5.3       | Dynamic DP                                        | 4         |
| <b>6</b>  | <b>Graph</b>                                      | <b>4</b>  |
| 6.1       | Max Clique                                        | 4         |
| 6.2       | Bellman-Ford                                      | 4         |
| 6.3       | System of Difference Constraints                  | 4         |
| 6.4       | Graph Girth                                       | 5         |
| 6.5       | Euler Trail                                       | 5         |
| 6.6       | Vertex BCC (Round Square Tree)                    | 5         |
| 6.7       | Edge BCC                                          | 5         |
| 6.8       | Kth Shortest Path                                 | 5         |
| 6.9       | SCC - Tarjan                                      | 6         |
| 6.10      | 2SAT                                              | 6         |
| <b>7</b>  | <b>Tree</b>                                       | <b>6</b>  |
| 7.1       | Tree Isomorphism (Rooted Trees)                   | 6         |
| 7.2       | Tree Isomorphism (Unrooted Trees)                 | 7         |
| 7.3       | Heavy Light Decomposition                         | 7         |
| 7.4       | Virtual Tree                                      | 7         |
| <b>8</b>  | <b>Matching</b>                                   | <b>7</b>  |
| 8.1       | Bipartite Matching                                | 7         |
| 8.2       | Bipartite Weighted Matching                       | 7         |
| 8.3       | General Matching                                  | 8         |
| 8.4       | General Weighted Matching                         | 8         |
| <b>9</b>  | <b>Flow</b>                                       | <b>9</b>  |
| 9.1       | Flow Methods                                      | 9         |
| 9.2       | Dinic (Flow Decomposition)                        | 10        |
| 9.3       | ISAP                                              | 10        |
| 9.4       | Bounded Max Flow                                  | 10        |
| 9.5       | MCMF                                              | 11        |
| 9.6       | Push-Relabel                                      | 11        |
| 9.7       | Gomory-Hu Tree                                    | 12        |
| 9.8       | Global Min Cut                                    | 12        |
| <b>10</b> | <b>String</b>                                     | <b>12</b> |
| 10.1      | Rolling Hash                                      | 12        |
| 10.2      | KMP                                               | 12        |
| 10.3      | Z Value                                           | 12        |
| 10.4      | Manacher                                          | 12        |
| 10.5      | Suffix Array + LCP                                | 12        |
| 10.6      | Suffix Automaton                                  | 13        |
| 10.7      | Minimum Rotation                                  | 13        |
| 10.8      | AC Automaton                                      | 13        |
| 10.9      | AC Automaton - Dict Links                         | 13        |
| 10.10     | AC Automaton - Counting Patterns                  | 13        |
| 10.11     | AC Automaton - Pattern Position                   | 13        |
| <b>11</b> | <b>Geometry</b>                                   | <b>13</b> |
| 11.1      | Template                                          | 13        |
| 11.2      | Basic Operations                                  | 14        |
| 11.3      | Lower Concave Hull                                | 14        |
| 11.4      | Pick's Theorem                                    | 14        |
| 11.5      | Minimum Enclosing Circle                          | 14        |
| 11.6      | PolyUnion                                         | 14        |
| 11.7      | Minkowski Sum                                     | 15        |
| <b>12</b> | <b>Number Theory</b>                              | <b>15</b> |
| 12.1      | Mod Sum                                           | 15        |
| 12.2      | Extended Lucas Theorem                            | 16        |
| 12.3      | Prime Sieve and Defactor                          | 16        |
| 12.4      | Harmonic Series                                   | 16        |
| 12.5      | Count Number of Divisors                          | 16        |
| 12.6      | 數論分塊                                              | 16        |
| 12.7      | Pollard's rho                                     | 17        |
| 12.8      | Miller Rabin                                      | 17        |
| 12.9      | Discrete Log                                      | 17        |
| 12.10     | Discrete Sqrt                                     | 17        |
| 12.11     | Fast Power                                        | 17        |
| 12.12     | Extend GCD                                        | 17        |
| 12.13     | Mu + Phi                                          | 18        |
| 12.14     | Other Formulas                                    | 18        |
| 12.15     | Polynomial                                        | 18        |
| 12.16     | Counting Primes                                   | 19        |
| 12.17     | Linear Sieve for Other Number Theoretic Functions | 20        |
| <b>13</b> | <b>Numerical</b>                                  | <b>20</b> |
| 13.1      | Polynomials and recurrences                       | 20        |
| 13.2      | Optimization                                      | 21        |
| 13.3      | Matrices                                          | 22        |
| 13.4      | Fourier transforms                                | 24        |
| <b>14</b> | <b>Combinatorics</b>                              | <b>24</b> |
| 14.1      | Catalan Number                                    | 24        |
| 14.2      | Bertrand's Ballot Theorem                         | 24        |
| 14.3      | Burnside's Lemma                                  | 24        |
| <b>15</b> | <b>Special Numbers</b>                            | <b>24</b> |
| 15.1      | Fibonacci Series                                  | 24        |
| 15.2      | Prime Numbers                                     | 24        |

|      |                        |    |
|------|------------------------|----|
| 15.3 | Number of Divisors     | 24 |
| 15.4 | Distinct Prime Factors | 24 |

## 1 Reminder

### 1.1 Observations and Tricks

- Contribution Technique
- 二分圖/Spanning Tree/DFS Tree
- 行、列操作互相獨立
- 奇偶性
- 當  $s, t$  遞增並且  $t = f(s)$ ，對  $s$  二分搜不好做，可以改成對  $t$  二分搜，再算  $f(t)$
- 啟發式合併
- Permutation Normalization (做一些平移對齊兩個 permutation)
- 枚舉  $a_1 \sim a_n$  再枚舉  $a_n \sim a_1$  可以包在一個迴圈
- 兩個凸型函數相加還是凸型函數，相減不一定
- 一個區間的  $\text{mex} = k$ ，表示這個區間包含  $0 \sim k-1$  所有數字，並且「 $U$  - 區間」的最小值  $= k$ 。

### 1.2 Bug List

- 沒開 long long
- 陣列戳出界 / 陣列開不夠大
- 寫好的函式 (例如 `init()`、`build()`) 忘記呼叫
- 0-base / 1-base
- 線段樹改值懶標初始值不能設為 0
- DFS 的時候不小心覆寫到全域變數
- 多筆測資不能沒讀完直接 return
- vector 超級肥，小 vector 請用 array，例如矩陣快速冪

## 2 Init (Linux)

### 2.1 vimrc

```
1syn on
2set nu rnu cul mouse=a cin et ts=2 sw=2 sts=2 autochdir
  cb=unnamedplus
3no ; :
4ino {<CR> {<CR>}<Esc>ko
5ca Hash w !cpp -dD -P \\\ tr -d '[:space:]' \\\ openssl sha256
  -r \\\ cut -c6
```

```
1" Gino's .vimrc "
2"no <C-h> ^ | <C-l> $ | h b | l e | b l | e h
```

### 2.2 bashrc

```
1gogo() {
2  g++ -std=c++20 -O2 "$1" && echo run && ./a.out
3}
4alias vig='vim -u ~/.vimrc-gino'
```

## 3 Basic

### 3.1 Template (Using Codebook)

```
1// CRyptogRapher
2#define SZ(c) ((int)(c).size())
3// kactl
4#define sz(x) (int)(x).size()
5#define rep(i,a,b) for(int i=(a);i<(b);i++)
6using ll = long long;
7using vi = vector<int>;
8using pii = pair<int, int>;
9// ckiseki
10#define all(x) begin(x), end(x)
11#define pb emplace_back
12#define REP(i, l, r) for (int i=(l); i<=(r); ++i)
13const ll LINF = 1e18;
14template<class A, class B> bool chmin(A& a, B& b) {
15  return b < a ? a = b, 1 : 0;
16}
```

### 3.2 PBDS, Random

```
1#include <bits/extc++.h>
2using namespace __gnu_pbds;
3// map
4tree<int, int, less<>, rb_tree_tag,
  tree_order_statistics_node_update> tr;
5tr.order_of_key(element);
6tr.find_by_order(rank);
7// set
8tree<int, null_type, less<>, rb_tree_tag,
  tree_order_statistics_node_update> tr;
9tr.order_of_key(element);
10tr.find_by_order(rank);
11// priority queue
12__gnu_pbds::priority_queue<int, less<int>> big_q;
13__gnu_pbds::priority_queue<int, greater<int>> small_q;
14ql.join(q2); // join
15
16mt19937
  gen(chrono::steady_clock::now().time_since_epoch().count());
17uniform_int_distribution<int>(a, b)(rng) // [a, b]
```

```
18 uniform_real_distribution<double>(a, b)(rng) // (a, b)
19 shuffle(v.begin(), v.end(), gen);
```

### 3.3 Debug

a265e1

```
1 #define debug(...) cerr << "\x1b[33m" #_VA_ARGS__ " : \
  \x1b[0m" , \
2   [ ](auto&&... a){ ((cerr << a << ' '), ...); }
  (__VA_ARGS__) , cerr << '\n'
3 #define output(...) \
4   [ ](auto&&... a){ ((cout << a << ' '), ...); }
  (__VA_ARGS__) , cout << '\n'
5 int v = 42;
6 format("{:b}", v); // "101010"
7 format("{:#b}", v); // "0b101010"
8 format("{:x}", v); // "0x2a"
9 format("{:X}", v); // "0X2A"
10 int x = 7;
11 format("{:05d}", x); // "00007"
12 format("{:>5}", x); // " 7"
13 format("{:<5}", x); // "7 "
14 format("{:^5}", x); // " 7 "
15 format("{:*^5}", x); // "***7***"
16 double pi = 3.1415926535;
17 format("{:.2f}", pi); // "3.14"
```

### 3.4 SVG Writer

```
1 // Author: ckiseki
2 class SVG {
3     void p(string_view s) { o << s; }
4     void p(string_view s, auto v, auto... vs) {
5         auto i = s.find('$');
6         o << s.substr(0, i) << v, p(s.substr(i + 1), vs...);
7     }
8     ofstream o; string c = "red";
9 public:
10    SVG(auto f, auto x1, auto y1, auto x2, auto y2) : o(f) {
11        p("<svg xmlns='http://www.w3.org/2000/svg' "
12          "viewBox='$ $ $ $'>\n"
13          "<style>{*stroke-width:0.5%;}</style>\n",
14          x1, -y2, x2 - x1, y2 - y1); }
15    ~SVG() { p("</svg>\n"); }
16    void color(string nc) { c = nc; }
17    void line(auto x1, auto y1, auto x2, auto y2) {
18        p("<line x1='$' y1='$' x2='$' y2='$' stroke='$'>\n",
19          x1, -y1, x2, -y2, c); }
20    void circle(auto x, auto y, auto r) {
21        p("<circle cx='$' cy='$' r='$' stroke='$' "
22          "fill='none'/>\n", x, -y, r, c); }
23    void text(auto x, auto y, string s, int w = 12) {
24        p("<text x='$' y='$' font-size='$px'>$</text>\n",
25          x, -y, w, s); }
26};

1 SVG svg("out.svg", -200, -200, 200, 200);
2 svg.color("lightgray"); // ↓ drawing x-y plane
3 svg.line(-200, 0, 200, 0); svg.line(0, -200, 0, 200);
4 svg.color("blue"); svg.line(10, 10, 80, 80);
5 svg.color("red"); svg.circle(50, 50, 1);
```

### 3.5 Python

00e632

```
1 import sys
2 input = sys.stdin.readline
3 readStr = lambda: input().rstrip('\r\n')
4 readInt = lambda: int(input())
5 readInts = lambda: list(map(int, input().split()))
6
7 from decimal import *
8 getcontext().prec = 50 # precision
9 x = Decimal(str(x))
10 y = Decimal("6.7")
11 x *= y
12 print(x.quantize(Decimal("0.000001"),
13   rounding=ROUND_HALF_EVEN))
14 # ROUND_CEILING : To INF (2.9 -> 3, -2.1 -> -2)
15 # ROUND_FLOOR : To -INF (2.1 -> 2, -2.9 -> -3)
16 # ROUND_HALF_EVEN: To nearest even (2.5 -> 2, 3.5 -> 4)
17 # ROUND_HALF_UP : Half away from 0 (2.5 -> 3, -2.5 -> -3)
18 # ROUND_HALF_DOWN: Half towards 0 (2.5 -> 2, -2.5 -> -2)
19 # ROUND_UP : Away from 0 (2.1 -> 3, -2.1 -> -3)
20 # ROUND_DOWN : Truncate to 0 (2.9 -> 2, -2.9 -> -2)
21 # <Default>: ROUND_HALF_EVEN
22 # <Standard School Math>: ROUND_HALF_UP
```

### 3.6 Stress Tests

```
1 from random import *
2 n = randint(1, 10)
3 arr = list(randint(1, 20) for i in range(n))
4 print(*arr)
5 # permutation / non-repetitive sequence
6 perm = sample(range(1, n + 1), n)
7 arr = sample(range(1, 10000 + 1), n)
```

```
8# string
9 from string import *
10 s = "".join(choices(ascii_lowercase, k=n))
11 t = "".join(choices(ascii_lowercase[:3], k=n)) # (a|b|c)*
12 u = "".join(choices(ascii_lowercase + ascii_uppercase +
13   digits, k=n))
14 # palindrome
15 half = "".join(choices(ascii_lowercase, k=(n + 1) // 2))
16 palindrome = half + (half[-2::-1] if n % 2 else half[::-1])
17 # ===== tree =====
18 n = 30
19 ## shallow tree
20 edges = [(randint(1, u - 1), u) for u in range(2, n + 1)]
21 ## deep tree
22 (suggested k: 2 ~ 5 or int(sqrt(n)) or 2n/D (D is
23   expected depth))
24 from math import *
25 k = int(sqrt(n))
26 edges = [(randint(max(1, u-k), u-1), u) for u in range(2, n
27   + 1)]
28 for e in edges:
29     print(*e)
30
31 edges = [(1, i) for i in range(2, n + 1)] ## star
32 edges = [(i, i + 1) for i in range(1, n)] ## chain
33 edges = [(i // 2, i) for i in range(2, n + 1)] ## complete
34   binary tree
35 # ===== graph =====
36 ## not necessarily connected, no multi-edge or self-loop
37 n = 10
38 density = 0.3 # graph density (0.0 ~ 1.0)
39 edges = [(i, j) for i in range(1, n+1) for j in range(i+1,
40   n+1) if random() < density]
41 ## connected, no multi-edge or self-loop
42 n, extra_edges = 10, 5
43 edges = set()
44 while len(edges) < n - 1 + extra_edges:
45     u, v = randint(1, n), randint(1, n)
46     if u != v:
47         edges.add((min(u, v), max(u, v)))
48 edges = list(edges)
49 ## DAG
50 perm = sample(range(1, n + 1), n)
51 edges = []
52 for _ in range(m):
53     i, j = sorted(sample(range(n), 2))
54     edges.append((perm[i], perm[j]))
55 ## Bipartite Graph
56 n1, n2 = 6, 7
57 left, right = list(range(1, n1+1)), list(range(n1+1,
58   n1+n2+1))
59 edges = [(choice(left), choice(right)) for _ in range(m)]
```

```
1 g++ -std=c++20 -o buggy $1
2 g++ -std=c++20 -o ac ac.cpp
```

```
3 for i in {1..100}; do
4     echo "TEST $i"
5     python3 gen.py > in
6     ./ac < in > oa
7     ./buggy < in > ob
8     if ! diff -q oa ob; then
9         cat in <(echo "=== AC") oa <(echo "=== WA") ob
10        break
11    fi
12 done
```

## 4 Data Structure

### 4.1 Mo's Algorithm

8d7194

```
1 // segments are 0-based
2 int B = max(1, (int)sqrt(n));
3 sort(Q.begin(), Q.end(), [&](const auto& a, const auto& b) {
4     if (a[0]/B != b[0]/B) return a[0]<b[0];
5     return (a[0]/B & 1) ? a[1]<b[1] : a[1]>b[1];
6 });
7 ll cur = 0; // current answer
8 int pl = 0, pr = -1;
9 for (auto& qi : Q) {
10    // get (l, r, qid) from qi
11    while (pl < l) del(pl++); while (pl > l) add(--pl);
12    while (pr < r) add(++pr); while (pr > r) del(pr--);
13    ans[qid] = cur;
14 }
```

### 4.2 CDQ

744a80

```
1 // preparation
2 // 1. write a 1-base BIT
3 // 2. init(n, mxc): n is number of elements, mxc = 值域
4 // 3. build info array CDQ.v by yourself (x, y, z, id)
5 // 4. call solve()
```

```

6// => cdq.ans[i] is number of j s.t. (xj <= xi, yj <= yi, zj
  <= zi) (j != i)
7struct CDQ {
8    struct info {
9        int x, y, z, id;
10        info(int x, int y, int z, int id):
11            x(x), y(y), z(z), id(id) {}
12        info():
13            x(0), y(0), z(0), id(0) {}
14    };
15    int n, mxc;
16    BIT bit;
17    vector<info> v;
18    vector<ll> ans;
19    void init(int _n, int _mxc) {
20        n = _n; mxc = _mxc;
21        v.assign(n, info{});
22        ans.assign(n, 0);
23    }
24    void dfs(int l, int r) {
25        if (l >= r) return;
26        int mid = (l+r) >> 1;
27        dfs(l, mid); dfs(mid+1, r);
28
29        vector<info> tmp;
30        vector<int> bit_op;
31        int pl = l, pr = mid+1;
32
33        while (pl <= mid && pr <= r) {
34            if (v[pl].y <= v[pr].y) {
35                tmp.emplace_back(v[pl]);
36                bit.upd(v[pl].z, 1);
37                bit_op.emplace_back(v[pl].z);
38                pl++;
39            }
40            else {
41                tmp.emplace_back(v[pr]);
42                ans[v[pr].id] += bit.qry(v[pr].z);
43                pr++;
44            }
45        }
46        while (pl <= mid) {
47            tmp.emplace_back(v[pl]);
48            pl++;
49        }
50        while (pr <= r) {
51            tmp.emplace_back(v[pr]);
52            ans[v[pr].id] += bit.qry(v[pr].z);
53            pr++;
54        }
55        for (int i = l, j = 0; i <= r; i++, j++) v[i] = tmp[j];
56        for (auto& op : bit_op) bit.upd(op, -1);
57    }
58    void solve() {
59        bit.init(mxc);
60        sort(v.begin(), v.end(), [&](const info& a, const info&
61        b){
62            return (a.x == b.x ? (a.y == b.y ? (a.z == b.z ?
63            a.id < b.id : a.z < b.z) : a.y < b.y) : a.x < b.x);
64        });
65        dfs(0, n-1);
66        map<pair<int, pii>, int> s;
67        sort(v.begin(), v.end(), [&](const info& a, const info&
68        b){
69            return a.id > b.id;
70        });
71        for (int i = 0, id = n-1; i < n; i++, id--) {
72            pair<int, pii> tmp = make_pair(v[i].x,
73            make_pair(v[i].y, v[i].z));
74            auto it = s.find(tmp);
75            if (it != s.end())
76                ans[id] += it->second;
77            s[tmp]++;
78        }
79    }
80};

```

### 4.3 Persistent Treap

803cce

```

1// Author: Ian
2struct node {
3    node *l, *r;
4    char c; int v, sz;
5    node(char x = '$'): c(x), v(mt()), sz(1) {
6        l = r = nullptr;
7    }
8    node(node* p) { *this = *p; }
9    void pull() {
10        sz = 1;
11        for (auto i : {l, r})
12            if (i) sz += i->sz;

```

```

13    }
14    arr[maxn], *ptr = arr;
15    inline int size(node* p) { return p ? p->sz : 0; }
16    node* merge(node* a, node* b) {
17        if (!a || !b) return a ? a : b;
18        if (a->v < b->v) {
19            node* ret = new(ptr++) node(a);
20            ret->r = merge(ret->r, b); ret->pull();
21            return ret;
22        }
23        else {
24            node* ret = new(ptr++) node(b);
25            ret->l = merge(a, ret->l); ret->pull();
26            return ret;
27        }
28    }
29    P<node*> split(node* p, int k) {
30        if (!p) return {nullptr, nullptr};
31        if (k >= size(p->l) + 1) {
32            auto [a, b] = split(p->r, k - size(p->l) - 1);
33            node* ret = new(ptr++) node(p);
34            ret->r = a; ret->pull();
35            return {ret, b};
36        }
37        else {
38            auto [a, b] = split(p->l, k);
39            node* ret = new(ptr++) node(p);
40            ret->l = b; ret->pull();
41            return {a, ret};
42        }
43    }

```

### 4.4 Li Chao Tree

6c33c2

```

1// Author: Ian
2// Function: For a set of lines L, find the maximum L_i(x)
  in L in O(lg n).
3typedef long double ld;
4constexpr int maxn = 5e4 + 5;
5struct line {
6    ld a, b;
7    ld operator()(ld x) { return a * x + b; }
8    arr[(maxn + 1) << 2];
9    bool operator<(line a, line b) { return a.a < b.a; }
10#define m ((l+r)>>1)
11void insert(line x, int i = 1, int l = 0, int r = maxn) {
12    if (r - l == 1) {
13        if (x(l) > arr[i](l))
14            arr[i] = x;
15        return;
16    }
17    line a = max(arr[i], x), b = min(arr[i], x);
18    if (a(m) > b(m))
19        arr[i] = a, insert(b, i << 1, l, m);
20    else
21        arr[i] = b, insert(a, i << 1 | 1, m, r);
22}
23ld query(int x, int i = 1, int l = 0, int r = maxn) {
24    if (x < l || r <= x) return -numeric_limits<ld>::max();
25    if (r - l == 1) return arr[i](x);
26    return max({arr[i](x), query(x, i << 1, l, m), query(x, i
  << 1 | 1, m, r)});
27}
28#undef m

```

### 4.5 Time Segment Tree

519775

```

1// Author: Ian
2struct Op; // reversible operation
3const int maxn=2e5+5;
4vector<Op> arr[(maxn+1)<<2];
5#define m ((l + r) >> 1)
6void insert(int ql, int qr, Op x, int i=1, int l=0, int
  r=maxn) {
7    if (qr<=l || r<=ql) return;
8    if (ql<=l && r<=qr) { arr[i].push_back(x); return; }
9    if (qr<=m)
10        insert(ql, qr, x, i<<1, l, m);
11    else if (m<=ql)
12        insert(ql, qr, x, i<<1|1, m, r);
13    else {
14        insert(ql, qr, x, i<<1, l, m);
15        insert(ql, qr, x, i<<1|1, m, r);
16    }
17}
18void traversal(vector<int>& ans, int i=1, int l=0, int
  r=maxn) {
19    int opcnt = 0;
20    for (auto val: arr[i])
21        if (DO_SOMETHING(val))
22            opcnt++;
23    if (r-l==1) ans[l] = CALC_ANS();

```

```

24     else {
25         traversal(ans, i<<1, l, m);
26         traversal(ans, i<<1|1, m, r);
27     }
28     while (opcnt--){
29         UNDO_SOMETHING();
30         arr[i].clear();
31     }
32 #undef m

```

## 5 DP

### 5.1 SOS DP

623a08

```

1 // Author: Gino
2 // Function: Solve problems that enumerates subsets of
3 // subsets (3^n => n*2^n)
4 for (int msk = 0; msk < (1<<n); msk++) {
5     for (int i = 1; i <= n; i++) {
6         if (msk & (1<<(i - 1))) {
7             dp[msk][i] = dp[msk][i - 1] + dp[msk ^ (1<<(i - 1))][i - 1];
8         } else {
9             dp[msk][i] = dp[msk][i - 1];
10        }
11    }
12 }

```

### 5.2 Divide and Conquer DP

d36c77

```

1 /* Author: Simon Lindholm (License: CC0)
2 * Description: Given $a[i] = \min_{lo(i) \le k < hi(i)} (f(i, k))$ where the (minimal)
3 * optimal $k$ increases with $i$, computes $a[i]$ for $i = L..R-i$.
4 * Time: $O((N + (hi-lo)) \log N) *$
5 struct DP { // Modify at will:
6     int lo(int ind) { return 0; }
7     int hi(int ind) { return ind; }
8     ll f(int ind, int k) { return dp[ind][k]; }
9     void store(int ind, int k, ll v) { res[ind] = pii(k, v); }
10
11 void rec(int L, int R, int LO, int HI) {
12     if (L >= R) return;
13     int mid = (L + R) >> 1;
14     pair<ll, int> best(LLONG_MAX, LO);
15     rep(k, max(LO, lo(mid)), min(HI, hi(mid)))
16         best = min(best, make_pair(f(mid, k), k));
17     store(mid, best.second, best.first);
18     rec(L, mid, LO, best.second+1);
19     rec(mid+1, R, best.second, HI);
20 }
21 void solve(int L, int R) { rec(L, R, INT_MIN, INT_MAX); }
22 };

```

### 5.3 Dynamic DP

- 適用情境：\$dp\_i = M\_i \cdot dp\_{i-1} \Rightarrow dp\_i = M\_i M\_{i-1} \dots M\_1 dp\_0\$
- 當 \$M\_i\$ 需要動態修改，且 \$M\_i\$ 是廣義矩陣乘法：

$$C_{ij} = \bigoplus_k A_{ik} \otimes B_{kj}$$

- 滿足 \$(\oplus, \otimes)\$ 是半環 (\$\otimes\$ 對 \$\oplus\$ 有分配律、\$\otimes\$ 和 \$\oplus\$ 有結合律、\$\oplus\$ 有交換律)
- 此時便可利用線段樹維護 \$M\_n M\_{n-1} \dots M\_1\$ 的乘積。線段樹上的 pos 是 \$M\_{pos}\$，但 pull 時要 \$st[idx] = mul(st[cr], st[cl])\$

## 6 Graph

### 6.1 Max Clique

3a1fe5

```

1 // max N = 64
2 typedef unsigned long long ll;
3 struct MaxClique{
4     ll nb[ N ], n, ans;
5     void init( ll _n ){
6         n = _n;
7         for( int i = 0 ; i < n ; i ++ ) nb[ i ] = 0LLU;
8     }
9     void add_edge( ll _u , ll _v ){
10         nb[ _u ] |= ( 1LLU << _v );
11         nb[ _v ] |= ( 1LLU << _u );
12     }
13 void B( ll r , ll p , ll x , ll cnt , ll res ){
14     if ( cnt + res < ans ) return;
15     if ( p == 0LLU && x == 0LLU ){
16         if ( cnt > ans ) ans = cnt;
17         return;
18     }
19     ll y = p | x; y &= -y;
20     ll q = p & ( ~nb[ int( log2( y ) ) ] );
21     while( q ){
22         ll i = int( log2( q & (-q) ) );
23         B( r | ( 1LLU << i ) , p & nb[ i ] , x & nb[ i ] , cnt + 1LLU , __builtin_popcountll( p & nb[ i ] ) );
24     }

```

```

25     q &= ~( 1LLU << i );
26     p &= ~( 1LLU << i );
27     x |= ( 1LLU << i );
28 }
29 }
30 int solve(){
31     ans = 0;
32     ll _set = 0;
33     if ( n < 64 ) _set = ( 1LLU << n ) - 1;
34     else{
35         for( ll i = 0 ; i < n ; i ++ ) _set |= ( 1LLU << i );
36     }
37     B( 0LLU , _set , 0LLU , 0LLU , n );
38     return ans;
39 }
40 }maxClique;

```

### 6.2 Bellman-Ford

92b21f

```

1 // Author: Gino
2 // n, m = #(vertices), #(edges), and vertices are 1-based
3 // 1. BellmanFord bf(G, n, m);
4 // 2-1: bf.calcDis(s);
5 // => bf.dis[u] := shortest dis from s to u
6 // => bf.dis[u] := LINF (s can't reach u)
7 // => bf.dis[u] := -LINF (dis[u] can be arbitrary small)
8 // 2-2: bf.findNegCycle();
9 // => return false (if no neg cycle)
10 // => return true (has neg cycle, and gives an example: bf.neg_cycle)
11 // Time: $O(VE)$, Space: $O(V + E)$
12 const ll LINF = 4e18;
13 struct BellmanFord {
14     const vector<vector<pair<int, ll>>> &G;
15     int n, m; // #(vertices), #(edges)
16     BellmanFord(const auto& G, int n, int m): G(G), n(n), m(m) {}
17     vector<ll> dis; vector<int> nth_relax, pa;
18     void run_bf(const auto& src) {
19         dis.assign(n+1, LINF); nth_relax.assign(n+1, 0); pa.assign(n+1, -1);
20         for (auto& s : src) dis[s] = 0;
21         for (int rlx = 1; rlx <= n; rlx++) {
22             for (int u = 1; u <= n; u++) {
23                 if (dis[u] == LINF) continue; // !important
24                 for (auto& [v, w] : G[u]) {
25                     if (dis[v] > dis[u] + w) {
26                         dis[v] = dis[u] + w; pa[v] = u;
27                         if (rlx == n) nth_relax[v] = 1;
28                     }
29                 }
30             }
31         }
32     }
33     void calcDis(int s) {
34         run_bf(vector<int>{s}); queue<int> q;
35         for (int u=1; u<=n; u++) if (nth_relax[u]) q.push(u);
36         while (!q.empty()) { int u = q.front(); q.pop();
37             for (auto& [v, w] : G[u]) {
38                 if (!nth_relax[v]) {
39                     nth_relax[v] = 1; q.push(v);
40                 }
41             }
42         }
43         for (int u=1; u<=n; u++) if (nth_relax[u]) dis[u] = -LINF;
44     }
45     vector<int> neg_cycle;
46     bool findNegCycle() {
47         auto src = views::iota(1, n + 1); run_bf(src);
48         auto it = ranges::find_if(src, [&](int s){ return nth_relax[s]; });
49         if (it == src.end()) return false;
50         int ptr = *it; for (int i=0; i<n; i++) ptr = pa[ptr];
51         neg_cycle.clear(); int cur = ptr;
52         while (true) {
53             neg_cycle.emplace_back(cur);
54             if (cur == ptr && neg_cycle.size() > 1) break;
55             cur = pa[cur];
56         }
57         ranges::reverse(neg_cycle);
58         return true;
59     }
60 };

```

### 6.3 System of Difference Constraints

fe55f9

```

1 vector<vector<pair<int, ll>>> G;
2 void add(int u, int v, ll w) { G[u].push_back({v, w}); }

```

- \$x\_u - x\_v \leq c \Rightarrow \text{add}(v, u, c)\$
- \$x\_u - x\_v \geq c \Rightarrow \text{add}(u, v, -c)\$
- \$x\_u - x\_v = c \Rightarrow \text{add}(v, u, c), \text{add}(u, v, -c)\$
- \$x\_u \geq c \Rightarrow \text{add super vertex } x\_0 = 0, \text{ then } x\_u - x\_0 \geq c \Rightarrow \text{add}(u, 0, -c)\$
- Don't forget non-negative constraints for every variable if specified implicitly.
- Interval sum \$\Rightarrow\$ Use prefix sum to transform into differential constraints. Don't forget \$S\_{i+1} - S\_i \geq 0\$ if \$x\_i\$ needs to be non-negative.



•  $x_u/x_v \leq c \Rightarrow \log x_u - \log x_v \leq \log c$

## 6.4 Graph Girth

Run BFS for every node, when encountered non-BFS-tree edge, update answer (min cycle length) with  $\text{dis}[u] + \text{dis}[v] + 1$ . Time  $O(VE)$ .

## 6.5 Euler Trail

11c393

```
1// Author: kactl (Simon Lindholm), wrapped by Gino
2// Vertices are 1-based, Edge ID are 0-based
3// 1. EulerWalk euler(n, true(directed) /
  false(undirected));
4// 2. euler.addEdge(u, v);
5// 3. int result = euler.solve();
6// => -1: nothing ( euler.path, euler.path_edges = {} )
7// => 1: Euler trail,
8// => 2: Euler circuit
9//     euler.path = {u_1, ..., u_{m+1}}
10//     euler.path_edges = {e_1, ..., e_m}
11// Time:  $O(V + E)$ , Space:  $O(V + E)$ 
12// Test:  $V, E \leq 2e5, 99ms$ 
13struct EulerWalk {
14    int n, m = 0; bool dir;
15    vector<vector<pii>> G; vi deg;
16    EulerWalk(int n, bool dir) : n(n), dir(dir), G(n + 1),
    deg(n + 1) {}
17    void addEdge(int u, int v) {
18        G[u].push_back({v, m});
19        if (!dir) G[v].push_back({u, m});
20        deg[u]++, deg[v] += dir ? -1 : 1;
21        m++;
22    }
23    vi path, path_edges;
24    void walk(int src) {
25        path.clear(); path_edges.clear();
26        vi its(n + 1), visE(m);
27        function<void(int, int)> dfs = [&](int u, int e_in) {
28            while (its[u] < sz(G[u])) {
29                auto [v, e] = G[u][its[u]++];
30                if (visE[e]) continue;
31                visE[e] = 1;
32                dfs(v, e);
33            }
34            path.push_back(u);
35            if (e_in != -1) path_edges.push_back(e_in);
36        };
37        dfs(src, -1);
38        if (sz(path) != m + 1) {
39            path.clear(); path_edges.clear(); return;
40        }
41        ranges::reverse(path); ranges::reverse(path_edges);
42    }
43    int solve() {
44        int s = -1, odd = 0, src = 0, dst = 0;
45        for (int u = 1; u <= n; u++) {
46            if (!dir && (deg[u] & 1)) odd++, s = u;
47            if (dir && deg[u] == 1) s = u, src++;
48            if (dir && deg[u] == -1) dst++;
49            if (dir && abs(deg[u]) > 1) return -1;
50        }
51        if (!dir && odd != 0 && odd != 2) return -1;
52        if (dir && !((src == 0 && dst == 0) || (src == 1 && dst == 1))) return -1;
53        if (s == -1) for (int u = 1; u <= n; u++) if (!
54            G[u].empty()) { s = u; break; }
55        if (s == -1) s = 1;
56        walk(s); return path.empty() ? -1 : path.front() ==
57        path.back() ? 2 : 1;
58    }
59};
```

## 6.6 Vertex BCC (Round Square Tree)

e85b0a

```
1// Author: Ian, https://judge.yosupo.jp/submission/399308
2// Vertex BCC (Round Square Tree) 0-based; Edge ID 0-based
3// 1. BCC bcc(n); // n = #(vertices)
4// 2. bcc.addEdge(u, v);
5// 3. bcc.build();
6// => bcc[id]: vertices in bcc `id` (square node n+id in T)
7// => T[id]: tree adjacency; square nodes are n..n+ncnt-1
8// => is_cut(x): whether x is a cut vertex
9// => bcc_id(u, v): square node containing tree-edge (u, v)
10struct BCC {
11    int n, m, t, ncnt;
12    vector<vector<pii>> G;
13    vector<vector<int>> T, bcc;
14    vector<int> dep, low, pa2, dep2, s;
15    void addEdge(int u, int v) {
16        G[u].eb(v, m), G[v].eb(u, m++);
17    }
18    void dfs(int x, int f) {
19        if (~dep[x]) return;
20        dep[x] = low[x] = t++;
21        for (auto [y, e]: G[x]) if (e != f) {
22            dfs(y, e), low[x] = min(low[x], low[y]);
23            if (low[y] >= dep[x]) {
24                int id = n + ncnt++;
25                bcc.resize(bcc.size()+1);
26                do {
27                    p = s.back(), s.pop_back();
28                    T[id].push_back(p), T[p].push_back(id);
29                    bcc.back().push_back(p);
30                } while (p != y);
31                T[id].push_back(x), T[x].push_back(id);
32                bcc.back().push_back(x);
33            } else low[x] = min(low[x], dep[y]);
34        }
35    }
36    void dfs2(int x, int p) {
37        dep2[x] = ~p ? dep2[p] + 1 : 0, pa2[x] = p;
38        for (auto y: T[x]) if (y != p) dfs2(y, x);
39    }
40    bool is_cut(int x) { return (int)T[x].size() != 1; }
41    int bcc_id(int u, int v) {
42        return pa2[dep2[u] < dep2[v] ? v : u] - n;
43    }
44    void build() {
45        for (int i = 0; i < n; i++) if (dep[i] == -1) {
46            if (G[i].empty()) { // isolated vertex: singleton bcc
47                dep[i] = low[i] = t++;
48                int id = n + ncnt++;
49                bcc.push_back({i});
50                T[id].push_back(i), T[i].push_back(id);
51            } else dfs(i, -1);
52        }
53        dfs2(0, -1);
54    }
55    BCC(int n): n(n), m(0), t(0), ncnt(0), G(n), T(2*n),
56    dep(n, -1), low(n), pa2(2*n), dep2(2*n) {}
57};
```

```
s.push_back(x);
for (auto [y, e]: G[x]) if (e != f) {
    if (dep[y] == -1) {
        dfs(y, e), low[x] = min(low[x], low[y]);
        if (low[y] >= dep[x]) {
            int id = n + ncnt++;
            bcc.resize(bcc.size()+1);
            do {
                p = s.back(), s.pop_back();
                T[id].push_back(p), T[p].push_back(id);
                bcc.back().push_back(p);
            } while (p != y);
            T[id].push_back(x), T[x].push_back(id);
            bcc.back().push_back(x);
        } else low[x] = min(low[x], dep[y]);
    }
}
void dfs2(int x, int p) {
    dep2[x] = ~p ? dep2[p] + 1 : 0, pa2[x] = p;
    for (auto y: T[x]) if (y != p) dfs2(y, x);
}
bool is_cut(int x) { return (int)T[x].size() != 1; }
int bcc_id(int u, int v) {
    return pa2[dep2[u] < dep2[v] ? v : u] - n;
}
void build() {
    for (int i = 0; i < n; i++) if (dep[i] == -1) {
        if (G[i].empty()) { // isolated vertex: singleton bcc
            dep[i] = low[i] = t++;
            int id = n + ncnt++;
            bcc.push_back({i});
            T[id].push_back(i), T[i].push_back(id);
        } else dfs(i, -1);
    }
    dfs2(0, -1);
}
BCC(int n): n(n), m(0), t(0), ncnt(0), G(n), T(2*n),
dep(n, -1), low(n), pa2(2*n), dep2(2*n) {}
};
```

## 6.7 Edge BCC

26729d

```
1// Author: Ian, https://judge.yosupo.jp/submission/399295
2// Edge BCC, 0-based; Edge ID 0-based
3// 1. EBCC ebcc(n); // n = #(vertices)
4// 2. ebcc.addEdge(u, v);
5// 3. ebcc.build();
6// => bcc[i]: each edge bcc
7// => bccn[x]: in which bcc is x
8// => isbr[e]: whether edge e is a bridge
9struct EBCC {
10    int n, m, t, ncnt;
11    vector<vector<pii>> G;
12    vector<vector<int>> bcc;
13    vector<bool> isbr;
14    vector<int> dep, low, bccn, s;
15    EBCC(int n): n(n), m(0), t(0), ncnt(0), G(n),
16    dep(n, -1), low(n), bccn(n) {}
17    void addEdge(int u, int v) {
18        G[u].eb(v, m), G[v].eb(u, m++);
19    }
20    void dfs(int x, int f) {
21        if (~dep[x]) return;
22        dep[x] = low[x] = t++;
23        s.push_back(x);
24        for (auto [y, e]: G[x]) if (e != f) {
25            if (dep[y] == -1)
26                dfs(y, e), low[x] = min(low[x], low[y]);
27            else
28                low[x] = min(low[x], dep[y]);
29        }
30        if (dep[x] == low[x]) {
31            if (~f) isbr[f] = true;
32            bcc.resize(bcc.size()+1);
33            int p; do {
34                bccn[p = s.back()] = ncnt;
35                s.pop_back();
36                bcc.back().push_back(p);
37            } while (p != x); ncnt++;
38        }
39    }
40    void build() {
41        isbr.assign(m, false);
42        for (int i = 0; i < n; i++) dfs(i, -1);
43    }
44};
```

## 6.8 Kth Shortest Path

34bacd

```
1// time:  $O(|E| \lg |E| + |V| \lg |V| + K)$ 
2// memory:  $O(|E| \lg |E| + |V|)$ 
3struct KSP { // 1-base
4    struct nd {
5        int u, v; ll d;
6        nd(int ui=0, int vi=0, ll di=INF) { u=ui; v=vi; d=di; }
7    };
8    struct heap { nd* edge; int dep; heap* chd[4]; };
9    static int cmp(heap* a, heap* b)
```

```

10 { return a->edge->d > b->edge->d; }
11 struct node{
12     int v; ll d; heap* H; nd* E;
13     node(){}
14     node(ll _d,int _v,nd* _E){ d=_d; v=_v; E=_E; }
15     node(heap* _H,ll _d){ H=_H; d=_d; }
16     friend bool operator<(node a,node b)
17     { return a.d>b.d; }
18 };
19 int n,k,s,t,dst[N]; nd *nxt[N];
20 vector<nd*> g[N],rg[N]; heap *nullNd,*head[N];
21 void init(int _n,int _k,int _s,int _t){
22     n=_n; k=_k; s=_s; t=_t;
23     for(int i=1;i<=n;i++){
24         g[i].clear(); rg[i].clear();
25         nxt[i]=NULL; head[i]=NULL; dst[i]=-1;
26     }
27 }
28 void addEdge(int ui,int vi,ll di){
29     nd* e=new nd(ui,vi,di);
30     g[ui].push_back(e); rg[vi].push_back(e);
31 }
32 queue<int> dfsQ;
33 void dijkstra(){
34     while(dfsQ.size()) dfsQ.pop();
35     priority_queue<node> Q; Q.push(node(0,t,NULL));
36     while (!Q.empty()){
37         node p=Q.top(); Q.pop(); if(dst[p.v]==-1)continue;
38         dst[p.v]=p.d; nxt[p.v]=p.E; dfsQ.push(p.v);
39         for(auto e:rg[p.v]) Q.push(node(p.d+e->d,e->u,e));
40     }
41 }
42 heap* merge(heap* curNd,heap* newNd){
43     if(curNd==nullNd) return newNd;
44     heap* root=new heap; memcpy(root,curNd,sizeof(heap));
45     if(newNd->edge->d<curNd->edge->d){
46         root->edge=newNd->edge;
47         root->chd[2]=newNd->chd[2];
48         root->chd[3]=newNd->chd[3];
49         newNd->edge=curNd->edge;
50         newNd->chd[2]=curNd->chd[2];
51         newNd->chd[3]=curNd->chd[3];
52     }
53     if(root->chd[0]->dep<root->chd[1]->dep)
54         root->chd[0]=merge(root->chd[0],newNd);
55     else root->chd[1]=merge(root->chd[1],newNd);
56     root->dep=max(root->chd[0]->dep,
57                 root->chd[1]->dep)+1;
58     return root;
59 }
60 vector<heap*> V;
61 void build(){
62     nullNd=new heap; nullNd->dep=0; nullNd->edge=new nd;
63     fill(nullNd->chd,nullNd->chd+4,nullNd);
64     while(not dfsQ.empty()){
65         int u=dfsQ.front(); dfsQ.pop();
66         if(!nxt[u]) head[u]=nullNd;
67         else head[u]=head[nxt[u]->v];
68         V.clear();
69         for(auto& e:g[u]){
70             int v=e->v;
71             if(dst[v]==-1) continue;
72             e->d+=dst[v]-dst[u];
73             if(nxt[u]!=e){
74                 heap* p=new heap; fill(p->chd,p->chd+4,nullNd);
75                 p->dep=1; p->edge=e; V.push_back(p);
76             }
77         }
78         if(V.empty()) continue;
79         make_heap(V.begin(),V.end(),cmp);
80 #define L(X) ((X<<1)+1)
81 #define R(X) ((X<<1)+2)
82         for(size_t i=0;i<V.size();i++){
83             if(L(i)<V.size()) V[i]->chd[2]=V[L(i)];
84             else V[i]->chd[2]=nullNd;
85             if(R(i)<V.size()) V[i]->chd[3]=V[R(i)];
86             else V[i]->chd[3]=nullNd;
87         }
88         head[u]=merge(head[u],V.front());
89     }
90 }
91 vector<ll> ans;
92 void first_K(){
93     ans.clear(); priority_queue<node> Q;
94     if(dst[s]==-1) return;
95     ans.push_back(dst[s]);
96     if(head[s]!=nullNd)
97         Q.push(node(head[s],dst[s]+head[s]->edge->d));

```

```

98     for(int _=1;_<k and not Q.empty();_++){
99         node p=Q.top();q=Q.pop(); ans.push_back(p.d);
100         if(head[p.H->edge->v]!=nullNd){
101             q.H=head[p.H->edge->v]; q.d=p.d+q.H->edge->d;
102             Q.push(q);
103         }
104         for(int i=0;i<4;i++){
105             if(p.H->chd[i]!=nullNd){
106                 q.H=p.H->chd[i];
107                 q.d=p.d-p.H->edge->d+p.H->chd[i]->edge->d;
108                 Q.push(q);
109             }
110         }
111     }
112     void solve(){ // ans[i] stores the i-th shortest path
113         dijkstra(); build();
114         first_K(); // ans.size() might less than k
115     }
116 } solver;

```

## 6.9 SCC - Tarjan

7a063d

```

1// Author: Ian, https://judge.yosupo.jp/submission/399291
2// Tarjan SCC, 0-based
3// 1. SCC scc(n); // n = #(vertices)
4// 2. Fill in adj list G
5// 3. scc.build()
6// => scc[i]: each scc
7// => sccn[x]: in which scc is x
8struct SCC {
9     const int inf = 1e9+5;
10     int n, t, ncnt;
11     vector<vector<int>> G, scc;
12     vector<bool> ins;
13     vector<int> sccn, dfn, low, s;
14     SCC(int n): n(n), t(0), ncnt(0), G(n),
15         ins(n, false), sccn(n), dfn(n, -1), low(n, inf) {}
16     void dfs(int x) {
17         if (~dfn[x]) return;
18         dfn[x] = low[x] = t++;
19         s.push_back(x), ins[x] = true;
20         for (auto i: G[x])
21             if (dfs(i), ins[i])
22                 low[x] = min(low[x], low[i]);
23         if (dfn[x] == low[x]) {
24             scc.resize(scc.size()+1);
25             int p; do {
26                 ins[p = s.back()] = false;
27                 s.pop_back(); sccn[p] = ncnt;
28                 scc.back().push_back(p);
29             } while (p != x); ncnt++;
30         }
31     }
32     void build() { for (int i=0; i<n; i++) dfs(i); }
33 };

```

## 6.10 2SAT

130f351

```

1// Author: Ian, CRYPTOGRAPHER
2struct TwoSAT: SCC {
3     TwoSAT(int n): SCC(2*n) {}
4     // (var a == na) v (var b == nb)
5     void add_disjunction(int a,int na,int b,int nb) {
6         a = 2*a^na, b = 2*b^nb;
7         G[a^1].push_back(b);
8         G[b^1].push_back(a);
9     }
10     vector<int> solve() {
11         build();
12         vector<int> assignment(n/2,0);
13         for(int i=0; i<n/2; i++) {
14             if (sccn[2*i] == sccn[2*i^1]) return {};
15             assignment[i] = sccn[2*i] > sccn[2*i^1];
16         }
17         return assignment;
18     }
19 };

```

## 7 Tree

### 7.1 Tree Isomorphism (Rooted Trees)

f3f029

```

1// Author: Gino
2// Checks if 2 rooted trees are isomorphic
3// Time: O(V log V)
4// vector<vector<int>> G1, G2;
5// int root_1, root_2;
6map<vector<int>, int> dict;
7int encode(const auto& G, int u, int pa = -1) {
8     vector<int> codes;
9     for (auto& v : G[u])
10         if (v != pa)
11             codes.emplace_back(encode(G, v, u));
12     ranges::sort(codes);
13     auto [it, _] = dict.try_emplace(codes, dict.size());

```

```

14 return it->second;
15 }
16 bool isomorphic() {
17     dict.clear(); dict[{}] = 0;
18     return encode(G1, root_1) == encode(G2, root_2);
19 }

```

## 7.2 Tree Isomorphism (Unrooted Trees)

Find the centroid(s) of  $T_1, T_2$ .

Case 1:  $T_1, T_2$  have different number of centroids  $\rightarrow$  NO

Case 2:  $T_1$  has centroid  $c_1, T_2$  has centroid  $c_2$

$\rightarrow$  rooted\_isomorphic( $c_1, c_2$ )

Case 3:  $T_1$  has centroids  $c_1, c_1', T_2$  has centroids  $c_2, c_2'$

$\rightarrow$  rooted\_isomorphic( $c_1, c_2$ ) || rooted\_isomorphic( $c_1', c_2'$ )

## 7.3 Heavy Light Decomposition

53e45c

```

1 // Author: Ian, https://judge.yosupo.jp/submission/399317
2 void segbuild(vector<int>&v);
3 void segmodify(int p, int k);
4 int segquery(int ql, int qr);
5 // Insert [ql, qr) segment tree here
6 template<class T=int> struct HLD {
7     int n, cnt;
8     vector<int> d, f, sz, son, top, dfn;
9     HLD(int n): n(n), cnt(0), d(n, 0), f(n, 0), sz(n, 1),
10         son(n, -1), top(n, 0), dfn(n, -1) {}
11     void dfs1(int x, int p) {
12         for (auto i: e[x]) if (i != p) {
13             d[i] = d[x] + 1, f[i] = x;
14             dfs1(i, x), sz[x] += sz[i];
15             if (son[x] == -1 || sz[son[x]] < sz[i])
16                 son[x] = i;
17         }
18     }
19     void dfs2(int x, int t) {
20         dfn[x] = cnt++; top[x] = t;
21         if (son[x] == -1) return;
22         dfs2(son[x], t);
23         for (auto i: e[x])
24             if (!dfn[i]) dfs2(i, i);
25     }
26     void build(const vector<T>&v) {
27         dfs1(0, 0); dfs2(0, 0);
28         vector<T> dfnv(v.size());
29         for (int i=0; i<n; i++)
30             dfnv[dfn[i]] = v[i];
31         segbuild(dfnv);
32     }
33     void modify(int p, T v) { segmodify(dfn[p], v); }
34     T query(int a, int b) {
35         auto agr = [](int x, int y) { return max(x, y); };
36         T ret = 0;
37         while (top[a] != top[b]) {
38             if (d[top[a]] > d[top[b]]) swap(a, b);
39             ret = agr(ret, segquery(dfn[top[b]], dfn[b]+1));
40             b = f[top[b]];
41         }
42         if (dfn[a] > dfn[b]) swap(a, b);
43         return agr(ret, segquery(dfn[a], dfn[b]+1));
44     }
45 };

```

## 7.4 Virtual Tree

172166

```

1 // Author: std_abc
2 // need lca, in, out
3 vector<pii> virtual_tree(vector<int>&v) {
4     auto cmp = [&](int x, int y) { return in[x] < in[y]; };
5     sort(all(v), cmp);
6     for (int i = 1; i < sz(v); ++i)
7         v.pb(lca(v[i-1], v[i]));
8     sort(all(v), cmp);
9     v.resize(unique(all(v)) - v.begin());
10    vector<int> stk(1, v[0]);
11    vector<pii> res;
12    for (int i = 1; i < sz(v); ++i) {
13        int x = v[i];
14        while (out[stk.back()] < out[x]) stk.pop_back();
15        res.emplace_back(stk.back(), x), stk.pb(x);
16    }
17    return res;
18 }

```

## 8 Matching

### 8.1 Bipartite Matching

e34126

```

1 // Author: Gino
2 // Function: Max Bipartite Matching in  $O(V \sqrt{E})$ 
3 // Usage:
4 // >>> init(nx, ny) -> add(x, y (+nx))
5 // >>> hk.max_matching() := the matching plan stores in mx, my
6 // >>> hk.min_vertex_cover() := the vertex cover plan stores in vcover

```

```

7 // (!) vertices are 0-based: X = [0, nx), Y = [nx, nx+ny)
8 #define pb emplace_back
9 struct HopcroftKarp {
10     int n, nx, ny;
11     vector<vector<int>> > G;
12     vector<int> mx, my;
13     void init(int _nx, int _ny) {
14         nx = _nx, ny = _ny;
15         n = nx + ny;
16         G.assign(n, vector<int>());
17     }
18     void add(int x, int y) { G[x].pb(y); G[y].pb(x); }
19     int max_matching() {
20         vector<int> dis, vis;
21         mx.assign(n, -1); my.assign(n, -1);
22         function<bool(int)> dfs = [&](int x) {
23             vis[x] = 1;
24             for (int y : G[x]) {
25                 int p = my[y];
26                 if (p == -1 || (dis[p] == dis[x] + 1 && !vis[p] &&
27                     dfs(p)))
28                     return mx[x] = y, my[y] = x, true;
29             }
30             return false;
31         };
32         while (true) {
33             dis.assign(n, -1);
34             queue<int> q;
35             for (int x = 0; x < nx; x++)
36                 if (mx[x] == -1) dis[x] = 0, q.push(x);
37             while (!q.empty()) {
38                 int x = q.front(); q.pop();
39                 for (int y : G[x])
40                     if (my[y] != -1 && dis[my[y]] == -1)
41                         dis[my[y]] = dis[x] + 1, q.push(my[y]);
42             }
43             vis.assign(n, 0);
44             bool found = false;
45             for (int x = 0; x < nx; x++)
46                 if (mx[x] == -1 && dfs(x)) found = true;
47             if (!found) break;
48         }
49         int ans = 0;
50         for (int x = 0; x < nx; x++) if (mx[x] != -1) ans++;
51         return ans;
52     }
53     vector<int> vcover;
54     int min_vertex_cover() {
55         int ans = max_matching();
56         vector<int> vis(n, 0);
57         function<void(int)> dfs = [&](int x) {
58             vis[x] = true;
59             for (int y : G[x]) {
60                 if (y == mx[x] || my[y] == -1 || vis[y]) continue;
61                 vis[y] = true;
62                 dfs(my[y]);
63             }
64         };
65         for (int x = 0; x < nx; x++) if (mx[x] == -1) dfs(x);
66         vcover.clear();
67         for (int x = 0; x < nx; x++) if (!vis[x]) vcover.pb(x);
68         for (int y = nx; y < nx + ny; y++) if (vis[y])
69             vcover.pb(y);
70         return ans;
71     }
72 };

```

### 8.2 Bipartite Weighted Matching

5933aa

```

1 // Author: ckiseki
2 // n = #(left vertices) = #(right vertices)
3 // left, right vertices labeled 1 ~ n separately
4 // 1. build 1-based matrix G of size (n + 1) * (n + 1)
5 // G[u][v] := weight between (left u, right v)
6 // <!> if not need perfect matching => set weights of all
7 // negative edges to 0
8 // 2. constrcut (solve at same time): KM km(n, G);
9 // => km.ans (max perfect weight sum)
10 // => km.fl[u] (left u -> right v)
11 // => km.fr[v] (right v -> left u)
12 // Time:  $O(V^3)$ , Space:  $O(V^2)$ 
13 // Test: V <= 500, 243ms
14 struct KM { // maximize, test @ UOJ 80
15     int n, l, r; ll ans; // fl and fr are the match
16     vector<ll> hl, hr; vector<int> fl, fr, pre, q;
17     void bfs(const auto &w, int s) {
18         vector<int> vl(n + 1), vr(n + 1); vector<ll> slk(n + 1,
19             LINf);
20         l = r = 0; vr[q[r++] = s] = true;
21         auto check = [&](int x) -> bool {

```

```

20     if (vl[x] || slk[x] > 0) return true;
21     vl[x] = true; slk[x] = LINF;
22     if (fl[x] != -1) return (vr[q[r++] = fl[x]] = true);
23     while (x != -1) swap(x, fr[fl[x] = pre[x]]);
24     return false;
25 };
26 while (true) {
27     while (l < r)
28         for (int x = l, y = q[l++]; x <= n; ++x) if (!vl[x])
29             if (ll val = hl[x] + hr[y] - w[x][y]; val <
30                 slk[x]) {
31                 slk[x] = val;
32                 if (pre[x] = y, !check(x)) return;
33             }
34     ll d = LINF;
35     for (int x = 1; x <= n; ++x) d = min(d, slk[x]);
36     for (int x = 1; x <= n; ++x)
37         vl[x] ? hl[x] += d : slk[x] -= d;
38     for (int x = 1; x <= n; ++x) if (vr[x]) hr[x] -= d;
39     for (int x = 1; x <= n; ++x) if (!check(x)) return;
40 }
41 KM(int n_, const auto &w) : n(n_), ans(0),
42     hl(n + 1), hr(n + 1), fl(n + 1, -1), fr(fl), pre(n + 1),
43     q(n + 1) {
44     for (int i = 1; i <= n; ++i) {
45         hl[i] = -LINF;
46         for (int j = 1; j <= n; ++j) hl[i] = max(hl[i],
47             (ll)w[i][j]);
48     }
49     for (int i = 1; i <= n; ++i) bfs(w, i);
50     for (int i = 1; i <= n; ++i) ans += w[i][fl[i]];
51 }

```

### 8.3 General Matching

93f948

```

1 // Author: ckiseki
2 // 0. build 1-based graph vector<vector<int>> G(n + 1);
3 // 1. construct (solve at same time):
4 //     GeneralMatching M(G, n); => M.ans
5 // status: M.match[x] (== -1 if not matched, 1 <= x <= n)
6 // Time: O(V^3), Space: O(V + E)
7 // Test: V <= 500, 11ms
8 struct GeneralMatching {
9     queue<int> q; int ans, n;
10    vector<int> fa, s, v, pre, match;
11    int Find(int u) {
12        return u == fa[u] ? u : fa[u] = Find(fa[u]);
13    }
14    int LCA(int x, int y) {
15        static int tk = 0; tk++; x = Find(x); y = Find(y);
16        for (; swap(x, y)) if (x != 0) {
17            if (v[x] == tk) return x;
18            v[x] = tk;
19            x = Find(pre[match[x]]);
20        }
21    }
22    void Blossom(int x, int y, int l) {
23        for (; Find(x) != l; x = pre[y]) {
24            pre[x] = y, y = match[x];
25            if (s[y] == 1) q.push(y), s[y] = 0;
26            for (int z : {x, y}) if (fa[z] == z) fa[z] = l;
27        }
28    }
29    bool Bfs(auto &&g, int r) {
30        iota(all(fa), 0); ranges::fill(s, -1);
31        q = queue<int>(); q.push(r); s[r] = 0;
32        for (; !q.empty(); q.pop()) {
33            for (int x = q.front(); int u : g[x])
34                if (s[u] == -1) {
35                    if (pre[u] = x, s[u] = 1, match[u] == 0) {
36                        for (int a = u, b = x, last;
37                            b != 0; a = last, b = pre[a])
38                            last = match[b], match[b] = a, match[a] = b;
39                        return true;
40                    }
41                    q.push(match[u]); s[match[u]] = 0;
42                } else if (!s[u] && Find(u) != Find(x)) {
43                    int l = LCA(u, x);
44                    Blossom(x, u, l); Blossom(u, x, l);
45                }
46            }
47        return false;
48    }
49    GeneralMatching(auto &&g, int n_) : ans(0), n(n_),
50        fa(n + 1), s(n + 1), v(n + 1), pre(n + 1, 0), match(n + 1,
51        0) {
52        for (int x = 1; x <= n; ++x)
53            if (match[x] == 0) ans += Bfs(g, x);
54        for (int x = 1; x <= n; ++x)

```

```

53         if (match[x] == 0) match[x] = -1;
54     }
55 }; // tested @ yosupo judge
56
57 8.4 General Weighted Matching 9660ba
58
59 1 // Author: ckiseki
60 2 // 1. GeneralWeightedMatching M(n);
61 3 //     (1-based, 1 <= u, v <= n)
62 4 // 2. add edges: M.set_edge(u, v, w);
63 5 // 3. auto [tot_weight, n_matches] = M.solve();
64 6 // 4. M.match[u] (== -1 if not matched, 1-based index if
65     matched)
66 7 // Time: O(V^3), Space: O(V^2)
67 8 // Test: V <= 500, 369ms
68 struct GeneralWeightedMatching { // 1-based
69     static const int inf = INT_MAX;
70     struct edge { int u, v, w; }; int n, nx;
71     vector<int> lab; vector<vector<edge>> g;
72     vector<int> slack, match, st, pa, S, vis;
73     vector<vector<int>> flo, flo_from; queue<int> q;
74     GeneralWeightedMatching(int n_) : n(n_), nx(n * 2), lab(nx
75         + 1),
76         g(nx + 1, vector<edge>(nx + 1)), slack(nx + 1),
77         flo(nx + 1, flo_from(nx + 1, vector(n + 1, 0))) {
78         match = st = pa = S = vis = slack;
79         REP(u, 1, n) REP(v, 1, n) g[u][v] = {u, v, 0};
80     }
81     int ED(edge e) {
82         return lab[e.u] + lab[e.v] - g[e.u][e.v].w * 2;
83     }
84     void update_slack(int u, int x, int &s) {
85         if (!s || ED(g[u][x]) < ED(g[s][x])) s = u;
86     }
87     void set_slack(int x) {
88         slack[x] = 0;
89         REP(u, 1, n)
90             if (g[u][x].w > 0 && st[u] != x && S[st[u]] == 0)
91                 update_slack(u, x, slack[x]);
92     }
93     void q_push(int x) {
94         if (x <= n) q.push(x);
95         else for (int y : flo[x]) q.push(y);
96     }
97     void set_st(int x, int b) {
98         st[x] = b;
99         if (x > n) for (int y : flo[x]) set_st(y, b);
100    }
101    vector<int> split_flo(auto &f, int xr) {
102        auto it = find(all(f), xr);
103        if (auto pr = it - f.begin(); pr % 2 == 1)
104            reverse(1 + all(f), it = f.end() - pr);
105        auto res = vector(f.begin(), it);
106        return f.erase(f.begin(), it), res;
107    }
108    void set_match(int u, int v) {
109        match[u] = g[u][v].v;
110        if (u <= n) return;
111        int xr = flo_from[u][g[u][v].u];
112        auto &f = flo[u], z = split_flo(f, xr);
113        REP(i, 0, int(z.size()) - 1) set_match(z[i], z[i ^ 1]);
114        set_match(xr, v); f.insert(f.end(), all(z));
115    }
116    void augment(int u, int v) {
117        for (;) {
118            int xnv = st[match[u]]; set_match(u, v);
119            if (!xnv) return;
120            set_match(v = xnv, u = st[pa[xnv]]);
121        }
122    }
123    /* SPLIT_HASH_HERE */
124    int lca(int u, int v) {
125        static int t = 0; ++t;
126        for (++t; u || v; swap(u, v)) if (u) {
127            if (vis[u] == t) return u;
128            vis[u] = t; u = st[match[u]];
129            if (u) u = st[pa[u]];
130        }
131        return 0;
132    }
133    void add_blossom(int u, int o, int v) {
134        int b = int(find(n + 1 + all(st), 0) - begin(st));
135        lab[b] = 0, S[b] = 0; match[b] = match[o];
136        vector<int> f = {o};
137        for (int x : {u, v}) {
138            for (int y; x != o; x = st[pa[y]])
139                f.pb(x), f.pb(y = st[match[x]]), q_push(y);
140            reverse(1 + all(f));
141        }
142        flo[b] = f; set_st(b, b);
143        REP(x, 1, nx) g[b][x].w = g[x][b].w = 0;
144        REP(x, 1, n) flo_from[b][x] = 0;

```



```

83 for (int xs : flo[b]) {
84     REP(x, 1, nx)
85     if (g[b][x].w == 0 || ED(g[xs][x]) < ED(g[b][x]))
86         g[b][x] = g[xs][x], g[x][b] = g[x][xs];
87     REP(x, 1, n)
88     if (flo_from[xs][x]) flo_from[b][x] = xs;
89 }
90 set_slack(b);
91 }
92 void expand_blossom(int b) {
93     for (int x : flo[b]) set_st(x, x);
94     int xr = flo_from[b][g[b][pa[b]].u], xs = -1;
95     for (int x : split_flo(flo[b], xr)) {
96         if (xs == -1) { xs = x; continue; }
97         pa[xs] = g[x][xs].u; S[xs] = 1, S[x] = 0;
98         slack[xs] = 0; set_slack(x); q_push(x); xs = -1;
99     }
100     for (int x : flo[b])
101         if (x == xr) S[x] = 1, pa[x] = pa[b];
102         else S[x] = -1, set_slack(x);
103     st[b] = 0;
104 }
105 bool on_found_edge(const edge &e) {
106     if (int u = st[e.u], v = st[e.v]; S[v] == -1) {
107         int nu = st[match[v]]; pa[v] = e.u; S[v] = 1;
108         slack[v] = slack[nu] = 0; S[nu] = 0; q_push(nu);
109     } else if (S[v] == 0) {
110         if (int o = lca(u, v)) add_blossom(u, o, v);
111         else return augment(u, v), augment(v, u), true;
112     }
113     return false;
114 }
115 /* SPLIT_HASH_HERE */
116 bool matching() {
117     ranges::fill(S, -1); ranges::fill(slack, 0);
118     q = queue<int>();
119     REP(x, 1, nx) if (st[x] == x && !match[x])
120         pa[x] = 0, S[x] = 0, q_push(x);
121     if (q.empty()) return false;
122     for (;;) {
123         while (q.size()) {
124             int u = q.front(); q.pop();
125             if (S[st[u]] == 1) continue;
126             REP(v, 1, n)
127                 if (g[u][v].w > 0 && st[u] != st[v]) {
128                     if (ED(g[u][v]) != 0)
129                         update_slack(u, st[v], slack[st[v]]);
130                     else if (on_found_edge(g[u][v])) return true;
131                 }
132         }
133         int d = inf;
134         REP(b, n + 1, nx) if (st[b] == b && S[b] == 1)
135             d = min(d, lab[b] / 2);
136         REP(x, 1, nx)
137             if (int s = slack[x]; st[x] == x && s && S[x] <= 0)
138                 d = min(d, ED(g[s][x]) / (S[x] + 2));
139         REP(u, 1, n)
140             if (S[st[u]] == 1) lab[u] += d;
141             else if (S[st[u]] == 0) {
142                 if (lab[u] <= d) return false;
143                 lab[u] -= d;
144             }
145         REP(b, n + 1, nx) if (st[b] == b && S[b] >= 0)
146             lab[b] += d * (2 - 4 * S[b]);
147         REP(x, 1, nx)
148             if (int s = slack[x]; st[x] == x &&
149                 s && st[s] != x && ED(g[s][x]) == 0)
150                 if (on_found_edge(g[s][x])) return true;
151         REP(b, n + 1, nx)
152             if (st[b] == b && S[b] == 1 && lab[b] == 0)
153                 expand_blossom(b);
154     }
155     return false;
156 }
157 pair<ll, int> solve() {
158     ranges::fill(match, 0);
159     REP(u, 0, n) st[u] = u, flo[u].clear();
160     int w_max = 0;
161     REP(u, 1, n) REP(v, 1, n) {
162         flo_from[u][v] = (u == v ? u : 0);
163         w_max = max(w_max, g[u][v].w);
164     }
165     REP(u, 1, n) lab[u] = w_max;
166     int n_matches = 0; ll tot_weight = 0;
167     while (matching()) ++n_matches;
168     REP(u, 1, n) if (match[u] && match[u] < u)
169         tot_weight += g[u][match[u]].w;
170     REP(u, 1, n) if (match[u] == 0) match[u] = -1;

```

```

171     return make_pair(tot_weight, n_matches);
172 }
173 void set_edge(int u, int v, int w) {
174     g[u][v].w = g[v][u].w = w; }
175 };

```

## 9 Flow

### 9.1 Flow Methods

c5090e

```

1 // Author: CRYPTOGRAPHER (some modified by Gino)
2 Maximize  $c^T x$  subject to  $Ax \leq b, x \geq 0$ ;
3 with the corresponding symmetric dual problem,
4 Minimize  $b^T y$  subject to  $A^T y \geq c, y \geq 0$ .
5
6 Maximize  $c^T x$  subject to  $Ax \leq b$ ;
7 with the corresponding asymmetric dual problem,
8 Minimize  $b^T y$  subject to  $A^T y = c, y \geq 0$ .
9
10 Maximize  $\sum x$  subject to  $x_i + x_j \leq A_{ij}, x \geq 0$ ;
11  $\Rightarrow$  Maximize  $\sum x$  subject to  $x_i + x_j \leq A_{ij}$ ;
12  $\Rightarrow$  Minimize  $A^T y = \sum A_{ij} y_{ij}$  subject to for all  $v$ ,
13      $\sum_{i=v \text{ or } j=v} y_{ij} = 1, y_{ij} \geq 0$ 
14  $\Rightarrow$  possible optimal solution:  $y_{ij} = \{0, 0.5, 1\}$ 
15  $\Rightarrow$  Minimum Bipartite perfect matching/2 ( $V_1=X, V_2=X, E=A$ )
16
17 General Graph:
18  $|Max \text{ Ind. Set}| + |Min \text{ Vertex Cover}| = |V|$ 
19  $|Max \text{ Ind. Edge Set}| + |Min \text{ Edge Cover}| = |V|$ 
20 Bipartite Graph:
21  $|Max \text{ Ind. Set}| = |Min \text{ Edge Cover}|$ 
22  $|Max \text{ Ind. Edge Set}| = |Min \text{ Vertex Cover}|$ 
23
24 To reconstruct the minimum vertex cover, dfs from each
25 unmatched vertex on the left side and with unused edges
26 only. Equivalently, dfs from source with unused edges
27 only and without visiting sink. Then, a vertex is
28 chosen iff. it is on the left side and without visited
29 or on the right side and visited through dfs.
30
31 Minimum Weighted Bipartite Edge Cover:
32 Construct new bipartite graph with  $n+m$  vertices on each
33 side:
34 for each vertex  $u$ , duplicate a vertex  $u'$  on the other side
35 for each edge  $(u,v,w)$ , add edges  $(u,v,w)$  and  $(v',u',w)$ 
36 for each vertex  $u$ , add edge  $(u,u',2w)$  where  $w$  is min edge
37 connects to  $u$ 
38 then the answer is the minimum perfect matching of the new
39 graph (KM)
40
41 Maximum density subgraph ( $\sum W_e + \sum W_v$ ) /  $|V|$ 
42 Binary search on answer:
43 For a fixed  $D$ , construct a Max flow model as follow:
44 Let  $S$  be Sum of all weight (or inf)
45 1. from source to each node with cap =  $S$ 
46 2. For each  $(u,v,w)$  in  $E$ ,  $(u \rightarrow v, \text{cap}=w), (v \rightarrow u, \text{cap}=w)$ 
47 3. For each node  $v$ , from  $v$  to sink with cap =  $S + 2 * D - \deg[v] - 2 * (W \text{ of } v)$ 
48 where  $\deg[v] = \sum \text{weight of edge associated with } v$ 
49 If  $\text{maxflow} < S * |V|$ ,  $D$  is an answer.
50
51 Requiring subgraph: all vertex can be reached from source
52 with
53 edge whose cap > 0.
54
55 Maximum closed subgraph
56 1. connect source with positive weighted
57 vertex(capacity=weight)
58 2. connect sink with negative weighted vertex(capacity=-
59 weight)
60 3. make capacity of the original edges = inf
61 4. ans = sum(positive weighted vertex weight) - (max flow)
62
63 (Node-disjoint) Min DAG Path Cover (用最少路徑覆蓋所有點)
64 Node disjoint: 拆出來的路徑不能共用同一個點
65 將一個點  $u$  裂成  $u_+$  和  $u_-$ , 代表進入和出去
66 對於一條邊  $(u, v)$  建邊  $u_- \Rightarrow v_+$ 
67 1.  $+$ 點們和  $-$ 點們形成一張二分圖
68 2. 最差的答案是  $n$ , 代表每個點自己一個點就是一條路徑
69 3. 只要一組  $(u_-, v_+)$  匹配成功那對應到的答案恰好會  $-1$ 
70  $\ggg$  ans =  $n - \text{這張二分圖的最大匹配}$ 
71
72 General DAG Path Cover
73 跟 Node-disjoint 版本差在點可以被很多條路共用
74 建邊方式 Node-disjoint 差別在條件比較鬆
75 Node-disjoint:  $u_- \Rightarrow v_+$  只能在  $(u, v)$  有邊時建
76 General:  $u_- \Rightarrow v_+$  在  $u$  能走到  $v$  的時候就建
77
78 Dilworth Theorem
79 反鏈：一些節點的集合，滿足這些節點互相無法抵達
80 最大反鏈 = 最小 General DAG Path Cover

```

```

75
76 Min Cut reconstruction (after running max flow s -> t):
77 From s, BFS/DFS through residual edges with remaining
78 capacity > 0, marking reached vertices as visited.
79 The cut set is every original edge (u, v) with u visited
80 and v not visited; its capacity sum equals the max flow.
81
82 Bipartite Matching reconstruction (after running max flow
83 on source -> X -> Y -> sink model):
84 For each edge (x, y) between the X-side and Y-side, it is
85 part of the matching iff. its residual capacity is 0
86 (i.e. it is saturated).
87
88 Max/Min Flow with Lower Bound (Circulation Problem):
89 1. Construct super source S and super sink T.
90 2. For each edge (x, y, l, u), connect x -> y with capacity
91    u - l.
92 3. For each vertex v, let in(v) = (sum of incoming lower
93    bounds)
94    - (sum of outgoing lower bounds).
95 4. If in(v) > 0, connect S -> v with capacity in(v);
96    otherwise connect v -> T with capacity -in(v).
97 To maximize: connect t -> s with capacity inf (skip this
98 edge
99 for a pure circulation problem), let f = maxflow(S, T).
100 If
101 f != sum of in(v) over all in(v) > 0, no solution exists.
102 Otherwise maxflow(s, t) (found after adding t -> s) is
103 the answer.
104 To minimize: let f = maxflow(S, T) (without any s-t edge
105 yet).
106 Then connect t -> s with capacity inf and let f' be the
107 extra
108 flow pushed from S to T. If f + f' != sum of in(v) over
109 all
110 in(v) > 0, no solution exists. Otherwise f' is the
111 answer.
112 5. The flow on each original edge e is l_e + f_e, where f_e
113 is
114 e's flow in the constructed graph.

```

## 9.2 Dinic (Flow Decomposition)

b8e775

```

1 // Function: Max Flow + Flow Decomposition, O(V^2 E)
2 // init -> addEdge; vertices 0-based
3 // >>> flow() := return max flow
4 // >>> flow_decomposition() := return max flow + store
5 // decomposition in flow.D
6 struct Dinic{
7     static const int MXN=10000;
8     struct Edge{ int v, c, r, C; bool fw; };
9     int n, s, t, level[MXN]; vector<Edge> G[MXN];
10    void init(int n, int s, int t){
11        n=n; s=s; t=t;
12        for (int i=0; i<n; i++) G[i].clear();
13    }
14    void addEdge(int u, int v, int c){
15        G[u].push_back({v, c, (int)G[v].size(), c, true});
16        G[v].push_back({u, 0, (int)G[u].size()-1, 0, false});
17    }
18    bool BFS(){
19        for (int i=0; i<n; i++) level[i]=-1;
20        queue<int> que; que.push(s); level[s]=0;
21        while(!que.empty()){
22            int u=que.front(); que.pop();
23            for (auto &it:G[u]){
24                if(it.c>0&&level[it.v]==-1){
25                    level[it.v]=level[u]+1; que.push(it.v);
26                }
27            }
28            return level[t]!=-1;
29        }
30    int DFS(int u, int nf){
31        if(u==t) return nf;
32        int res=0;
33        for (auto &it:G[u]){
34            if(it.c>0&&level[it.v]==level[u]+1){
35                int tf=DFS(it.v, min(nf, it.c));
36                res+=tf; nf-=tf; it.c-=tf;
37                G[it.v][it.r].c+=tf;
38                if(nf==0) return res;
39            }
40        }
41        if(!res) level[u]=-1;
42        return res;
43    }
44    int flow(int res=0){
45        while(BFS()) res+=DFS(s, 2147483647);
46        return res;
47    } // reset: do nothing
48    vector<pair<int, vector<int>>> D; // (flow amount, [path
49    pl ... pk])
50    int flow_decomposition(){
51        int mxflow = flow();
52        vector<vector<Edge>> fG(n); // graph with forward edges
53        for (int u = 0; u < n; u++) {
54            for (auto &e : G[u]) {
55                if (e.fw && e.C - e.c > 0)

```

```

56                fG[u].push_back({e.v, e.C - e.c, e.r, e.C, e.fw});
57            }
58        }
59        vector<int> vis(n, 0);
60        function<int(int, int)> dfs = [&](int u, int cur) {
61            if (u == t) {
62                D.back().second.push_back(u);
63                return cur;
64            }
65            vis[u] = 1;
66            for (auto &e : fG[u]) {
67                if (e.c > 0 && !vis[e.v]) {
68                    int ans = dfs(e.v, min(cur, e.c));
69                    if (ans > 0) {
70                        e.c -= ans;
71                        D.back().second.push_back(u);
72                        return ans;
73                    }
74                }
75            }
76            return 0;
77        };
78        D.clear();
79        int quota = mxflow;
80        while (quota > 0) {
81            D.emplace_back(0, vector<int>());
82            fill(vis.begin(), vis.end(), 0);
83            int f = dfs(s, 2147483647);
84            if (f == 0) break;
85            reverse(D.back().second.begin(),
86                  D.back().second.end());
87            D.back().first = f, quota -= f;
88        }
89        return mxflow;
90    }
91 } flow;

```

## 9.3 ISAP

c13d55

```

1 // Author: CRYPTOGRAPHER
2 // 1-based: vertices are numbered 1 ~ n
3 // 1. flow.init(n, s, t);
4 // n = #vertices, s, t = source, sink
5 // 2. flow.addEdge(u, v, c); // u, v in [1, n]
6 // 3. call: int ans = flow.flow();
7 // => ans = max flow value from s to t
8 // Time: O(V^2 * E) worst case, usually much faster in
9 // practice
10 static const int MAXV=50010;
11 static const int INF =1000000;
12 struct Maxflow{
13     struct Edge{
14         int v, c, r;
15         Edge(int _v, int _c, int _r):v(_v), c(_c), r(_r){}
16     };
17     int s, t; vector<Edge> G[MAXV];
18     int iter[MAXV], d[MAXV], gap[MAXV], tot;
19     void init(int n, int s, int t){
20         tot=n, s=s, t=t;
21         for(int i=1; i<=tot; i++){
22             G[i].clear(); iter[i]=d[i]=gap[i]=0;
23         }
24     }
25     void addEdge(int u, int v, int c){
26         G[u].push_back(Edge(v, c, SZ(G[v])));
27         G[v].push_back(Edge(u, 0, SZ(G[u])-1));
28     }
29     int DFS(int p, int flow){
30         if(p==t) return flow;
31         for(int &i=iter[p]; i<SZ(G[p]); i++){
32             Edge &e=G[p][i];
33             if(e.c>0&&d[p]==d[e.v]+1){
34                 int f=DFS(e.v, min(flow, e.c));
35                 if(f){ e.c-=f; G[e.v][e.r].c+=f; return f; }
36             }
37         }
38         if(--gap[d[p]]==0) d[s]=tot;
39         else{ d[p]++; iter[p]=0; ++gap[d[p]]; }
40         return 0;
41     }
42     int flow(){
43         int res=0;
44         for(res=0, gap[0]=tot; d[s]<tot; res+=DFS(s, INF));
45         return res;
46     }
47     void reset(){
48         for(int i=1; i<=tot; i++) iter[i]=d[i]=gap[i]=0;
49     }
50 } flow;

```

## 9.4 Bounded Max Flow

aaf8c3

```

1 // Author: CRYPTOGRAPHER
2 // Max flow with lower/upper bound on edges (source-sink
3 // version)
4 // <!=> 1-based: original vertices are numbered 1 ~ n.
5 // Super source, sink: n+1, n+2 respectively.

```

```

5 // <!-- dependency: ISAP.cpp
6 // 1. n = #vertices (1 ~ n), m = #edges
7 //    s, t = original source, sink (both in [1, n])
8 // 2. l[i], r[i] := edge i goes from l[i] -> r[i] (l[i],
9 //    r[i] in [1, n])
10 // 3. a[i], b[i] := flow on edge i must lie in [a[i], b[i]]
11 // 4. int ans = solve(n, m, s, t);
12 // => ans == -1 : no feasible flow exists
13 // => ans : feasible max flow value
14 // <!-- N, M must be large enough for original graph + 2
15 //    super nodes
16 // Time: O(V^2 E), Space: O(V + E)
17 int in[N], out[N], l[M], r[M], a[M], b[M];
18 int solve(int n, int m, int s, int t){
19     flow.init(n+2, n+1, n+2); // super source = n+1, super
20     sink = n+2
21     for(int i = 0; i < m; i++){
22         in[r[i]] += a[i]; out[l[i]] += a[i];
23         flow.addEdge(l[i], r[i], b[i] - a[i]);
24         // flow from l[i] to r[i] must lie in [a[i], b[i]]
25     }
26     int nd = 0;
27     for(int i = 1; i <= n; i++){
28         if(in[i] < out[i]){
29             flow.addEdge(i, flow.t, out[i] - in[i]);
30             nd += out[i] - in[i];
31         }
32         if(out[i] < in[i])
33             flow.addEdge(flow.s, i, in[i] - out[i]);
34     }
35     flow.addEdge(t, s, INF); // original sink -> source
36     if(flow.flow() != nd) return -1; // infeasible
37     int ans = flow.G[s].back().c; // t->s edge's current flow
38     = feasible flow s->t
39     flow.G[s].back().c = flow.G[t].back().c = 0;
40     for(size_t i = 0; i < flow.G[flow.s].size(); i++){
41         Maxflow::Edge &e = flow.G[flow.s][i];
42         flow.G[flow.s][i].c = 0; flow.G[e.v][e.r].c = 0;
43     }
44     for(size_t i = 0; i < flow.G[flow.t].size(); i++){
45         Maxflow::Edge &e = flow.G[flow.t][i];
46         flow.G[flow.t][i].c = 0; flow.G[e.v][e.r].c = 0;
47     }
48     flow.addEdge(flow.s, s, INF); flow.addEdge(t, flow.t,
49     INF);
50     flow.reset(); // keeps G[] intact, only resets search
51     state
52     return ans + flow.flow();
53 }

```

## 9.5 MCMF

472be6

```

1 // Author: CRYPTOGRAPHER
2 // MCMF (supports negative edges without negative cycles)
3 // <!-- 1-based: vertices are numbered 1 ~ n.
4 // 1. MCMF.init(n); // n = #vertices (1 ~ n)
5 // 2. MCMF.add(u, v, cap, cost);
6 // 3. auto [max_flow, min_cost] = MCMF.flow(s, t); // s, t
7 //    in [1, n]
8 // Time: O(F * E), Space: O(V + E)
9 typedef int Tcost;
10 const int MAXV = 20010;
11 const int INFf = 1000000;
12 const Tcost INFc = 1e9;
13 struct MCMF {
14     struct Edge {
15         int v, cap;
16         Tcost w;
17         int rev;
18         bool fw;
19         Edge(){}
20         Edge(int t2, int t3, Tcost t4, int t5, bool t6)
21             : v(t2), cap(t3), w(t4), rev(t5), fw(t6) {}
22     };
23     int V, s, t;
24     vector<Edge> G[MAXV];
25     void init(int n){
26         V = n;
27         for(int i = 0; i <= V; i++) G[i].clear();
28     }
29     void add(int a, int b, int cap, Tcost w){
30         G[a].push_back(Edge(b, cap, w, (int)G[b].size(), true));
31         G[b].push_back(Edge(a, 0, -w, (int)G[a].size()-1,
32         false));
33     }
34     Tcost d[MAXV];
35     int id[MAXV], mom[MAXV];
36     bool inqu[MAXV];
37     queue<int> q;
38     pair<int, Tcost> flow(int _s, int _t){
39         s = _s, t = _t;

```

```

38     int mxf = 0; Tcost mnc = 0;
39     while(1){
40         fill(d, d+V, INFc); // need to use type cast
41         fill(inqu, inqu+V, 0);
42         fill(mom, mom+V, -1);
43         mom[s] = s;
44         d[s] = 0;
45         q.push(s); inqu[s] = 1;
46         while(q.size()){
47             int u = q.front(); q.pop();
48             inqu[u] = 0;
49             for(int i = 0; i < (int) G[u].size(); i++){
50                 Edge &e = G[u][i];
51                 int v = e.v;
52                 if(e.cap > 0 && d[v] > d[u]+e.w){
53                     d[v] = d[u]+e.w;
54                     mom[v] = u;
55                     id[v] = i;
56                     if(!inqu[v]) q.push(v), inqu[v] = 1;
57                 }
58             }
59         }
60         if(mom[t] == -1) break;
61         int df = INFf;
62         for(int u = t; u != s; u = mom[u])
63             df = min(df, G[mom[u]][id[u]].cap);
64         for(int u = t; u != s; u = mom[u]){
65             Edge &e = G[mom[u]][id[u]];
66             e.cap -= df;
67             G[e.v][e.rev].cap += df;
68         }
69         mxf += df;
70         mnc += df*d[t];
71     }
72     return make_pair(mxf, mnc);
73 }
74 }

```

## 9.6 Push-Relabel

fae237

```

1 // Author: Simon Lindholm (kactl)
2 // 0-based: vertices are numbered 0 ~ n-1
3 // 1. PushRelabel pr(n); // n = #vertices
4 // 2. pr.addEdge(u, v, c); // u, v in [0, n-1]
5 // 3. ll ans = pr.calc(s, t);
6 // => ans = max flow value from s to t
7 // To obtain actual flow:
8 // for (int u = 0; u < n; u++)
9 //     for (auto& e : pr.g[u])
10 //         if (e.f > 0) => (u, e.dest, e.f)
11 // Time: O(V^2 * sqrt(E)), better on dense graph
12 struct PushRelabel {
13     struct Edge { int dest, back; ll f, c; };
14     vector<vector<Edge>> g;
15     vector<ll> ec;
16     vector<Edge*> cur;
17     vector<vi> hs; vi H;
18     PushRelabel(int n) : g(n), ec(n), cur(n), hs(2*n), H(n) {}
19 };
20 void addEdge(int s, int t, ll cap, ll rcap=0) {
21     if (s == t) return;
22     g[s].push_back({t, sz(g[t]), 0, cap});
23     g[t].push_back({s, sz(g[s])-1, 0, rcap});
24 }
25 void addFlow(Edge& e, ll f) {
26     Edge &back = g[e.dest][e.back];
27     if (!ec[e.dest] && f) hs[H[e.dest]].push_back(e.dest);
28     e.f += f; e.c -= f; ec[e.dest] += f;
29     back.f -= f; back.c += f; ec[back.dest] -= f;
30 }
31 ll calc(int s, int t) {
32     int v = sz(g); H[s] = v; ec[t] = 1;
33     vi co(2*v); co[0] = v-1;
34     rep(i, 0, v) cur[i] = g[i].data();
35     for (Edge& e : g[s]) addFlow(e, e.c);
36 }
37 for (int hi = 0;;) {
38     while (hs[hi].empty()) if (!hi--) return -ec[s];
39     int u = hs[hi].back(); hs[hi].pop_back();
40     while (ec[u] > 0) // discharge u
41         if (cur[u] == g[u].data() + sz(g[u])) {
42             H[u] = 1e9;
43             for (Edge& e : g[u]) if (e.c && H[u] >
44             H[e.dest]+1)
45                 H[u] = H[e.dest]+1, cur[u] = &e;
46             if (++co[H[u]], !--co[hi] && hi < v)
47                 rep(i, 0, v) if (hi < H[i] && H[i] < v)
48                     --co[H[i]], H[i] = v + 1;
49             hi = H[u];
50         } else if (cur[u]->c && H[u] == H[cur[u]->dest]+1)
51             addFlow(*cur[u], min(ec[u], cur[u]->c));

```

```

51     else ++cur[u];
52 }
53 }
54 bool leftOfMinCut(int a) { return H[a] >= sz(g); }
55 };

```

### 9.7 Gomory-Hu Tree 88b014

```

1// Author: chilli, Takanori MAEHARA (kactl)
2// <!-- Dependency: PushRelabel.cpp
3// Given a list of edges representing an undirected flow
graph,
4// returns edges of the Gomory-Hu tree.
5// maxflow / mincut of (u, v) => min edge from u to v on
Gomory-Hu tree
6// Time: O(V) * O(PushRelabel)
7typedef array<ll, 3> Edge; // (u, v, w)
8vector<Edge> gomoryHu(int N, vector<Edge> ed) {
9    vector<Edge> tree;
10    vi par(N);
11    rep(i, 1, N) {
12        PushRelabel D(N); // Dinic also works
13        for (Edge t : ed) D.addEdge(t[0], t[1], t[2], t[2]);
14        tree.push_back({i, par[i], D.calc(i, par[i])});
15        rep(j, i+1, N)
16            if (par[j] == par[i] && D.leftOfMinCut(j)) par[j] = i;
17    }
18    return tree;
19 };

```

### 9.8 Global Min Cut a040ee

```

1// Author: ckiseki
2// 1-based: vertices are numbered 1 ~ n
3// 1. vector<vector<ll>> G(n+1, vector<ll>(n+1, 0));
4// 2. add_edge(G, u, v, c); // add undirected edge u-v with
weight c, u, v in [1, n]
5// 3. ll ans = mincut(G, n); // n = #vertices
6// => ans = global min cut value
7// Time: O(V^3)
8void add_edge(auto &w, int u, int v, int c) {
9    w[u][v] += c; w[v][u] += c; }
10auto phase(const auto &w, int n, vector<int> id) {
11    vector<ll> g(n+1); int s = -1, t = -1;
12    while (!id.empty()) {
13        int c = -1;
14        for (int i : id) if (c == -1 || g[i] > g[c]) c = i;
15        s = t; t = c;
16        id.erase(ranges::find(id, c));
17        for (int i : id) g[i] += w[c][i];
18    }
19    return tuple{s, t, g[t]};
20 }
21ll mincut(auto w, int n) {
22    ll cut = numeric_limits<ll>::max();
23    vector<int> id(n); iota(all(id), 1);
24    for (int i = 0; i < n - 1; ++i) {
25        auto [s, t, gt] = phase(w, n, id);
26        id.erase(ranges::find(id, t));
27        cut = min(cut, gt);
28        for (int j = 1; j <= n; ++j)
29            w[s][j] += w[t][j], w[j][s] += w[j][t];
30    }
31    return cut;
32 };

```

## 10 String

### 10.1 Rolling Hash c15659

```

1// Author: ckiseki (Modified by Gino)
2// Usage: RH<Sigma, Mod> Hs(s);
3// If no modifications needed, replace BIT with prefix sum
4template <ll P = 127, ll Q = 1051762951>
5class RH {
6    int n; vector<ll> p, inv, d, bit;
7    ll add(ll a, ll b) { return a+b >= Q ? a+b-Q : a+b; }
8    ll sub(ll a, ll b) { return a-b < 0 ? a-b+Q : a-b; }
9    ll mul(ll a, ll b) { return (__int128)a * b % Q; }
10    ll pw(ll a, ll x) {
11        ll r = 1; for (; x; a = mul(a, a), x >>= 1)
12            if (x & 1) r = mul(r, a);
13        return r;
14    }
15    void bit_add(int i, ll x)
16        { for (; i <= n; i += i&-i) bit[i] = add(bit[i], x); }
17    ll bit_qry(int i)
18        { ll s = 0; for (; i; i -= i&-i) s = add(s, bit[i]); }
19    public:
20    RH(const auto &s) : n(s.size()),
21        p(n+1, 1), inv(n+1, 1), d(n+1), bit(n+1) {
22        ll iv = pw(P, Q - 2);
23        for (int i = 1; i <= n; i++) {

```

```

24        p[i] = mul(p[i - 1], P); inv[i] = mul(inv[i - 1], iv);
25        d[i] = mul(s[i - 1], p[i]); bit_add(i, d[i]);
26    } }
27    void update(int i, char c)
28        { ll x=mul(c, p[i]); bit_add(i, sub(x, d[i])); d[i]=x; }
29    ll query(int l, int r)
30        { return mul(sub(bit_qry(r), bit_qry(l - 1)), inv[l]); }
31 };

```

### 10.2 KMP 376cdf

```

1// Author: Gino
2vector<int> kmp(auto s){ int n=s.size(); vector<int> f(n,0);
3    for (int i = 1, ptr = 0; i < n; i++) {
4        while (ptr && s[i] != s[ptr]) ptr = f[ptr - 1];
5        if (s[i] == s[ptr]) ptr++; f[i] = ptr;
6    } return f;
7 }

```

### 10.3 Z Value d5b3b9

```

1// Author: std_abs
2vector<int> Z(auto s){ int n=s.size(); vector<int> Z(n);
3    for (int i = 0, l = 0, r = 0; i < n; ++i) {
4        Z[i] = max(min(Z[i - l], r - i), 0);
5        while (i + Z[i] < n && s[Z[i]] == s[i + Z[i]])
6            l = i, r = i + Z[i], Z[i]++;
7    } return Z;
8 }

```

### 10.4 Manacher 193b08

```

1// Author: std_abs
2// return array Z (length == 2N - 1)
3// Example: s = "aaaa"
4//      a      a      a      a
5// Z_0 Z_1 Z_2 Z_3 Z_4 Z_5 Z_6
6// 1 2 3 4 3 2 1
7// The corresponding palindrome s[l..r]:
8// l = (i - Z[i] + 1) / 2
9// r = (i + Z[i] - 1) / 2
10vector<int> manacher(auto tmp) {
11    vector<int> s(1, -1);
12    for (auto c : tmp) s.push_back(c), s.push_back(-1);
13    int l = 0, r = 0, n = (int)s.size();
14    vector<int> Z(n);
15    for (int i = 0; i < n; ++i) {
16        Z[i] = r > i ? min(Z[2 * l - i], r - i) : 1;
17        while (i - Z[i] >= 0 && i + Z[i] < n &&
18            s[i + Z[i]] == s[i - Z[i]]) Z[i]++;
19        if (Z[i] + i > r) l = i, r = Z[i] + i;
20    }
21    for (int i = 0; i < n; ++i)
22        Z[i] = (Z[i] - (i & 1)) / 2 * 2 + (i & 1);
23    return vector<int>(Z.begin() + 1, Z.end() - 1);
24 }

```

### 10.5 Suffix Array + LCP c1df5b

```

1// Author: std_abs
2// Tested @yosupo, N <= 5e5, 123ms
3const int N = 5e5 + 10;
4int sa[N], tmp[2][N], c[N], rk[N], lcp[N];
5void buildSA(string s) {
6    int *x = tmp[0], *y = tmp[1], m = 256, n = s.size();
7    for (int i = 0; i < m; ++i) c[i] = 0;
8    for (int i = 0; i < n; ++i) c[x[i]] = s[i]++;
9    for (int i = 1; i < m; ++i) c[i] += c[i - 1];
10    for (int i = n - 1; ~i; --i) sa[--c[x[i]]] = i;
11    for (int k = 1; k < n; k <= 1) {
12        for (int i = 0; i < m; ++i) c[i] = 0;
13        for (int i = 0; i < n; ++i) c[x[i]]++;
14        for (int i = 1; i < m; ++i) c[i] += c[i - 1];
15        int p = 0;
16        for (int i = n - k; i < n; ++i) y[p++] = i;
17        for (int i = 0; i < n; ++i) if (sa[i] >= k)
18            y[p++] = sa[i] - k;
19        for (int i = n - 1; ~i; --i)
20            sa[--c[x[y[i]]]] = y[i];
21        y[sa[0]] = p = 0;
22        for (int i = 1; i < n; ++i) {
23            int a = sa[i], b = sa[i - 1];
24            p += !(x[a]==x[b] && a+k < n && b+k < n && x[a+k]==x[b+k]);
25            y[sa[i]] = p;
26        }
27        if (n == p + 1) break;
28        swap(x, y), m = p + 1;
29    } }
30    void buildLCP(string s) {
31        // lcp[i] = LCP(sa[i - 1], sa[i])
32        // lcp(i, j) = query_lcp_min[rk[i] + 1, rk[j] + 1)
33        int n = s.length(), x = 0;
34        for (int i = 0; i < n; ++i) rk[sa[i]] = i;
35        for (int i = 0; i < n; ++i) {

```



```

36 if (!rk[i]) lcp[rk[i]] = 0;
37 else { if (x) x--;
38         int p = sa[rk[i] - 1];
39         while (x+i<n && x+p<n && s[x+i]==s[x+p]) x++;
40         lcp[rk[i]] = x;
41 } } }

```

## 10.6 Suffix Automaton

10f751

```

1// Author: std_abs
2// state (i.e. node) -> all substr with the same endpos
3// those substrs' length in range [len(link) + 1, len]
4// state i's endpos set -> union of pos[i] over the subtree
  of i in the suffix link tree
5// link[i] -> longest suffix with different endpos set
  (suffix link tree edge)
6// len[i] -> length longest suffix of state i
7// pos[i] -> an example end position of state i (1-based)
8// cnt[i] -> size of endpos set of state i, built by
  build_cnt() (tree DP)
9const int N = 5e5 + 10, C = 26;
10struct SAM {
11    int ch[2*N][C], len[2*N], link[2*N], pos[2*N], cnt[2*N], _id;
12    SAM () { reset(); }
13    void reset() { _id = 0, newnode(), link[0] = -1; }
14    int newnode() {
15        fill_n(ch[_id], C, 0);
16        len[_id] = link[_id] = pos[_id] = cnt[_id] = 0;
17        return _id++;
18    }
19    void build(string s) {
20        int last = 0; for (int i = 0; i < (int)s.size(); i++) {
21            char c = s[i]; int cur = newnode();
22            len[cur] = len[last] + 1, pos[cur] = i + 1;
23            int p = last;
24            while (~p && !ch[p][c - 'a'])
25                ch[p][c - 'a'] = cur, p = link[p];
26            if (p == -1) link[cur] = 0;
27            else { // conflict: p --> q, p --> cur
28                int q = ch[p][c - 'a'];
29                if (len[p] + 1 == len[q]) { // q is suffix of cur
30                    link[cur] = q;
31                } else { // we need auxiliary state
32                    int nxt = newnode();
33                    len[nxt] = len[p] + 1, link[nxt] = link[q];
34                    pos[nxt] = 0;
35                    for (int j = 0; j < C; j++)
36                        ch[nxt][j] = ch[q][j];
37                    while (~p && ch[p][c - 'a'] == q)
38                        ch[p][c - 'a'] = nxt, p = link[p];
39                    link[q] = link[cur] = nxt;
40                }
41            }
42            cnt[cur]++, last = cur;
43        }
44    }
45    void build_cnt() {
46        // vector<int> p(_id); iota(all(p), 0); sort(all(p),
47        // [&](int i, int j){ return len[i] > len[j]; });
48        // for (int i = 0; i < _id; ++i) if (link[p[i]] != -1)
49        //     cnt[link[p[i]]] += cnt[p[i]];
50    }
51    bool match(string s) {
52        int p = 0; for (auto& c : s) {
53            if (!ch[p][c - 'a']) return false;
54            p = ch[p][c - 'a'];
55        } return true; }

```

## 10.7 Minimum Rotation

bb235f

```

1// Author: std_abs
2string rotate(auto s){ int n=s.size(), i=0, j=1;
3    string t = s + s;
4    while (i < n && j < n) {
5        int k = 0; while (k < n && t[i + k] == t[j + k]) ++k;
6        if (t[i + k] <= t[j + k]) j += k + 1; else i += k + 1;
7        if (i == j) ++j;
8    } return t.substr((i < n ? i : j), n);
9}

```

## 10.8 AC Automaton

285968

```

1// Author: std_abs (slightly modified by Gino)
2// Recap: fail[s] = s 的出現在 trie 上 (也就是某個 pattern 的前
  綴) 的最長 proper 後綴
3// N := sum(length of all patterns) + 1
4// 1. ac.insert(pat) -> end-node id recorded in ac.pid
5// 2. ac.build_fail()
6// 3. run text: ptr = to[ptr][c-'a'] each char; accumulate
  on your own arrays
7// - pull: rep(i ac.order_cnt-1 ~ 0) val[fail[x]] += val[x]
8// - push: rep(i 0 ~ ac.order_cnt-1) val[x] += val[fail[x]]
9// Tested @yosupo, N <= 1e6, 323ms on building AC
10const int N = 1e6 + 5, C = 26;
11struct AC {

```

```

12    int ch[N][C], to[N][C], fail[N], _id, order[N], order_cnt;
13    int np; vector<int> pid; // pattern ID on trie
14    // order[r] := which trie node has BFS order r
15    // fail link tree: (parent) fail[i] <-- i (child)
16    void reset() { _id = np = order_cnt = 0, pid.clear(),
  newnode(); }
17    int newnode() {
18        fill_n(ch[_id], C, 0), fill_n(to[_id], C, 0);
19        fail[_id] = 0; return _id++;
20    }
21    AC () { reset(); }
22    void insert(string s) {
23        int ptr = 0; for (char c : s) {
24            if (!ch[ptr][c - 'a']) ch[ptr][c - 'a'] = newnode();
25            ptr = ch[ptr][c - 'a'];
26        } np++; pid.emplace_back(ptr);
27    }
28    void build_fail() {
29        queue<int> q; order_cnt = 0;
30        for (int i = 0; i < C; ++i) if (ch[0][i])
31            q.push(ch[0][i]), to[0][i] = ch[0][i];
32        while (!q.empty()) {
33            int u = q.front(); q.pop(); order[order_cnt++] = u;
34            for (int i = 0; i < C; ++i) {
35                if (!ch[u][i]) to[u][i] = to[fail[u]][i];
36                else { int v = ch[u][i], k = fail[u];
37                    while (k && !ch[k][i]) k = fail[k];
38                    if (ch[k][i]) k = ch[k][i];
39                    fail[v] = k, to[u][i] = v;
40                    q.push(v);
41                }
42            }
43        }
44    }

```

## 10.9 AC Automaton - Dict Links

a62cfa

```

1// Author: Gino, paste this in `struct AC {...}`
2// 3 Modifications to the original AC Automaton:
3// int np; vector<int> pid; bool is_pattern[N];
4// fail[_id] = 0, is_pattern[_id] = false; return _id++;
5// np++; pid.emplace_back(ptr); is_pattern[ptr] = true;
6int dict[N];
7void build_dict() { // call this after build_fail()!
8    for (int i = 0; i < order_cnt; i++) {
9        int u = order[i], f = fail[u];
10        dict[u] = is_pattern[f] ? f : dict[f];
11    }
12}

```

## 10.10 AC Automaton - Counting Patterns

58fd32

```

1// Author: Gino, paste this in `struct AC {...}`
2ll val[N];
3void count_patterns(string& text, vector<string>& patterns){
4    for (auto& p : patterns) insert(p); build_fail();
5    ranges::fill(val, 0LL);
6    int ptr = 0; for (auto& c : text) {
7        ptr = to[ptr][c - 'a']; val[ptr]++;
8    }
9    for (int x : span(order, order_cnt) | views::reverse)
10        val[fail[x]] += val[x];
11    for (auto& i : pid) cout << val[i] << endl;
12}

```

## 10.11 AC Automaton - Pattern Position

輸出每個 pattern 在 text 中第一次出現的位置，若不存在則輸出 -1。  
走 AC 自動機時 if (val[ptr] == -1) val[ptr] = i; 接著 pull DP  
if (val[fail[x]] == -1) continue;  
if (val[fail[x]] == -1) val[fail[x]] = val[x];  
else val[fail[x]] = min(val[fail[x]], val[x]);

## 11 Geometry

### 11.1 Template

2229b3

```

1// Author: Gino
2typedef long long T;
3// typedef long double T;
4const long double eps = 1e-8;
5
6short sgn(T x) {
7    if (abs(x) < eps) return 0;
8    return x < 0 ? -1 : 1;
9}
10
11struct Pt {
12    T x, y;
13    Pt(T _x=0, T _y=0):x(_x), y(_y) {}
14    Pt operator+(Pt a) { return Pt(x+a.x, y+a.y); }
15    Pt operator-(Pt a) { return Pt(x-a.x, y-a.y); }
16    Pt operator*(T a) { return Pt(x*a, y*a); }
17    Pt operator/(T a) { return Pt(x/a, y/a); }
18    T operator*(Pt a) { return x*a.x + y*a.y; }
19    T operator^(Pt a) { return x*a.y - y*a.x; } // 不要打反
20    bool operator<(Pt a)

```

```

21 { return x < a.x || (x == a.x && y < a.y); }
22 //return sgn(x-a.x) < 0 || (sgn(x-a.x) == 0 && sgn(y-a.y) <
    0); }
23 bool operator==(Pt a)
24 { return sgn(x-a.x) == 0 && sgn(y-a.y) == 0; }
25 };
26
27 Pt mv(Pt a, Pt b) { return b-a; }
28 T len2(Pt a) { return a*a; }
29 T dis2(Pt a, Pt b) { return len2(b-a); }
30
31 short ori(Pt a, Pt b) { return ((a^b)>0) - ((a^b)<0); }
32 bool onseg(Pt p, Pt l1, Pt l2) {
33     Pt a = mv(p, l1), b = mv(p, l2);
34     return ((a^b) == 0) && ((a*b) <= 0);
35 }

```

## 11.2 Basic Operations

```

1 // Author: Gino
2 // Function: Check if a point P sits in a polygon (doesn't
    have to be convex hull)
3 // 0 = Bound, 1 = In, -1 = Out
4 short inPoly(Pt p) {
5     for (int i = 0; i < n; i++)
6         if (onseg(p, E[i], E[(i+1)%n])) return 0;
7     int cnt = 0;
8     for (int i = 0; i < n; i++)
9         if (banana(p, Pt(p.x+1, p.y+2e9), E[i], E[(i+1)%n]))
10             cnt ^= 1;
11     return (cnt ? 1 : -1);
12 }

1 // Author: Gino
2 int ud(Pt a) { // up or down half plane
3     if (a.y > 0) return 0;
4     if (a.y < 0) return 1;
5     return (a.x >= 0 ? 0 : 1);
6 }
7 sort(ALL(E), [&](const Pt& a, const Pt& b){
8     if (ud(a) != ud(b)) return ud(a) < ud(b);
9     return (a^b) > 0;
10 });

```

```

1 // Author: Gino
2 // Function: check if (p1---p2) (q1---q2) banana
3 inline bool banana(Pt p1, Pt p2, Pt q1, Pt q2) {
4     if (onseg(p1, q1, q2) || onseg(p2, q1, q2) ||
5         onseg(q1, p1, p2) || onseg(q2, p1, p2)) {
6         return true;
7     }
8     Pt p = mv(p1, p2), q = mv(q1, q2);
9     return (ori(p, mv(p1, q1)) * ori(p, mv(p1, q2)) < 0 &&
10         ori(q, mv(q1, p1)) * ori(q, mv(q1, p2)) < 0);
11 }

```

```

1 // Author: Gino
2 // T: long double
3 Pt bananaPoint(Pt p1, Pt p2, Pt q1, Pt q2) {
4     if (onseg(q1, p1, p2)) return q1;
5     if (onseg(q2, p1, p2)) return q2;
6     if (onseg(p1, q1, q2)) return p1;
7     if (onseg(p2, q1, q2)) return p2;
8     double s = abs(mv(p1, p2) ^ mv(p1, q1));
9     double t = abs(mv(p1, p2) ^ mv(p1, q2));
10    return q2 * (s/(s+t)) + q1 * (t/(s+t));
11 }

1 // Author: Gino
2 vector<Pt> hull;
3 void convexHull() {
4     hull.clear(); sort(E.begin(), E.end());
5     for (int t : {0, 1}) {
6         int b = (int)hull.size();
7         for (auto& ei : E) {
8             while ((int)hull.size() - b >= 2 &&
9                 ori(mv(hull[(int)hull.size()-2],
10                     mv(hull[(int)hull.size()-2], ei)) == -1)
11             {
12                 hull.pop_back();
13                 hull.emplace_back(ei);
14             }
15             hull.pop_back();
16             reverse(E.begin(), E.end());
17         }

```

```

1 // Author: Gino
2 // Function: Return doubled area of a polygon
3 T dbarea(vector<Pt>& e) {
4     ll res = 0;

```

```

5 for (int i = 0; i < (int)e.size(); i++)
6     res += e[i]^e[(i+1)%SZ(e)];
7 return abs(res);
8 }

```

## 11.3 Lower Concave Hull

2a96ed

```

1 // Author: Unknown
2 struct Line {
3     mutable ll m, b, p;
4     bool operator<(const Line& o) const { return m < o.m; }
5     bool operator<(ll x) const { return p < x; }
6 };
7
8 struct LineContainer : multiset<Line, less<>> {
9     // (for doubles, use inf = 1/.0, div(a,b) = a/b)
10    const ll inf = LLONG_MAX;
11    ll div(ll a, ll b) { // floored division
12        return a / b - ((a ^ b) < 0 && a % b); }
13    bool isect(iterator x, iterator y) {
14        if (y == end()) { x->p = inf; return false; }
15        if (x->m == y->m) x->p = x->b > y->b ? inf : -inf;
16        else x->p = div(y->b - x->b, x->m - y->m);
17        return x->p >= y->p;
18    }
19    void add(ll m, ll b) {
20        auto z = insert({m, b, 0}), y = z++, x = y;
21        while (isect(y, z)) z = erase(z);
22        if (x != begin() && isect(--x, y)) isect(x, y =
23            erase(y));
24        while ((y = x) != begin() && (--x)->p >= y->p)
25            isect(x, erase(y));
26    }
27    ll query(ll x) {
28        assert(!empty());
29        auto l = *lower_bound(x);
30        return l.m * x + l.b;
31    }

```

## 11.4 Pick's Theorem

Consider a polygon which vertices are all lattice points.  
 Let  $i$  = number of points inside the polygon.  
 Let  $b$  = number of points on the boundary of the polygon.  
 Then we have the following formula:

$$\text{Area} = i + b/2 - 1$$

## 11.5 Minimum Enclosing Circle

533348

```

1 // Author: Gino
2 // Function: Find Min Enclosing Circle using Randomized O(n)
    Algorithm
3 Pt circumcenter(Pt A, Pt B, Pt C) {
4     // a1(x-A.x) + b1(y-A.y) = c1
5     // a2(x-A.x) + b2(y-A.y) = c2
6     // solve using Cramer's rule
7     T a1 = B.x-A.x, b1 = B.y-A.y, c1 = dis2(A, B)/2.0;
8     T a2 = C.x-A.x, b2 = C.y-A.y, c2 = dis2(A, C)/2.0;
9     T D = Pt(a1, b1) ^ Pt(a2, b2);
10    T Dx = Pt(c1, b1) ^ Pt(c2, b2);
11    T Dy = Pt(a1, c1) ^ Pt(a2, c2);
12    if (D == 0) return Pt(-INF, -INF);
13    return A + Pt(Dx/D, Dy/D);
14 }
15 Pt center; T r2;
16
17 void minEncloseCircle() {
18     mt19937
19     gen(chrono::steady_clock::now().time_since_epoch().count());
20     shuffle(ALL(E), gen);
21     center = E[0], r2 = 0;
22
23     for (int i = 0; i < n; i++) {
24         if (dis2(center, E[i]) <= r2) continue;
25         center = E[i], r2 = 0;
26         for (int j = 0; j < i; j++) {
27             if (dis2(center, E[j]) <= r2) continue;
28             center = (E[i] + E[j]) / 2.0;
29             r2 = dis2(center, E[i]);
30             for (int k = 0; k < j; k++) {
31                 if (dis2(center, E[k]) <= r2) continue;
32                 center = circumcenter(E[i], E[j], E[k]);
33                 r2 = dis2(center, E[i]);
34             }
35         }

```

## 11.6 PolyUnion

673fdd

```

1 // Author: Unknown
2 struct PY{
3     int n; Pt pt[5]; double area;
4     Pt& operator[](const int x){ return pt[x]; }

```

```

5 void init(){ //n,pt[0~n-1] must be filled
6   area=pt[n-1]^pt[0];
7   for(int i=0;i<n-1;i++) area+=pt[i]^pt[i+1];
8   if((area/=2)<0)reverse(pt,pt+n),area=-area;
9 }
10};
11PY py[500]; pair<double,int> c[5000];
12inline double segP(Pt &p,Pt &p1,Pt &p2){
13   if(dcmp(p1.x-p2.x)==0) return (p.y-p1.y)/(p2.y-p1.y);
14   return (p.x-p1.x)/(p2.x-p1.x);
15}
16double polyUnion(int n){ //py[0~n-1] must be filled
17   int i,j,ii,jj,ta,tb,r,d; double z,w,s,sum=0,tc,td;
18   for(i=0;i<n;i++) py[i][py[i].n]=py[i][0];
19   for(i=0;i<n;i++){
20     for(ii=0;ii<py[i].n;ii++){
21       r=0;
22       c[r++]=make_pair(0.0,0); c[r++]=make_pair(1.0,0);
23       for(j=0;j<n;j++){
24         if(i==j) continue;
25         for(jj=0;jj<py[j].n;jj++){
26           ta=dcmp(tri(py[i][ii],py[i][ii+1],py[j][jj]));
27           tb=dcmp(tri(py[i][ii],py[i][ii+1],py[j][jj+1]));
28           if(ta==0 && tb==0){
29             if((py[j][jj+1]-py[j][jj])*(py[i][ii+1]-py[i][ii]))>0&&j<i){
30               c[r++]=make_pair(segP(py[j][jj],py[i][ii],py[i][ii+1]),1);
31               c[r++]=make_pair(segP(py[j][jj+1],py[i][ii],py[i][ii+1]),-1);
32             }
33             else if(ta>=0 && tb<0){
34               tc=tri(py[j][jj],py[j][jj+1],py[i][ii]);
35               td=tri(py[j][jj],py[j][jj+1],py[i][ii+1]);
36               c[r++]=make_pair(tc/(tc-td),1);
37             }
38             else if(ta<0 && tb>=0){
39               tc=tri(py[j][jj],py[j][jj+1],py[i][ii]);
40               td=tri(py[j][jj],py[j][jj+1],py[i][ii+1]);
41               c[r++]=make_pair(tc/(tc-td),-1);
42             }
43             sort(c,c+r);
44             z=min(max(c[0].first,0.0),1.0); d=c[0].second; s=0;
45             for(j=1;j<r;j++){
46               w=min(max(c[j].first,0.0),1.0);
47               if(!d) s+=w-z;
48               d+=c[j].second; z=w;
49             }
50             sum+=(py[i][ii]^py[i][ii+1])*s;
51           }
52         }
53       return sum/2;
54     }
55 }

```

## 11.7 Minkowski Sum

7762c5

```

1// Author: Unknown
2/* convex hull Minkowski Sum*/
3#define INF 10000000000000LL
4int pos( const Pt& tp ){
5   if( tp.Y == 0 ) return tp.X > 0 ? 0 : 1;
6   return tp.Y > 0 ? 0 : 1;
7}
8#define N 300030
9Pt pt[ N ], qt[ N ], rt[ N ];
10LL Lx,Rx;
11int dn,un;
12inline bool cmp( Pt a, Pt b ){
13   int pa=pos( a ),pb=pos( b );
14   if(pa==pb) return (a^b)>0;
15   return pa<pb;
16}
17int minkowskiSum(int n,int m){
18   int i,j,r,p,q,fi,fj;
19   for(i=1,p=0;i<n;i++){
20     if( pt[i].Y<pt[p].Y ||
21        (pt[i].Y==pt[p].Y && pt[i].X<pt[p].X) ) p=i; }
22   for(i=1,q=0;i<m;i++){
23     if( qt[i].Y<qt[q].Y ||
24        (qt[i].Y==qt[q].Y && qt[i].X<qt[q].X) ) q=i; }
25   rt[0]=pt[p]+qt[q];
26   r=1; i=p; j=q; fi=fj=0;
27   while(1){
28     if((fj&&j==q) ||
29        ( (!fi||i==p) &&
30          cmp(pt[(p+1)%n]-pt[p],qt[(q+1)%m]-qt[q]) ) ){
31       rt[r]=rt[r-1]+pt[(p+1)%n]-pt[p];
32       p=(p+1)%n;
33       fi=1;
34     }else{
35       rt[r]=rt[r-1]+qt[(q+1)%m]-qt[q];
36       q=(q+1)%m;

```

```

37       fj=1;
38     }
39     if(r<=1 || ((rt[r]-rt[r-1])^(rt[r-1]-rt[r-2]))!=0) r++;
40     else rt[r-1]=rt[r];
41     if(i==p && j==q) break;
42   }
43   return r-1;
44}
45void initInConvex(int n){
46   int i,p,q;
47   LL Ly,Ry;
48   Lx=INF; Rx=-INF;
49   for(i=0;i<n;i++){
50     if(pt[i].X<Lx) Lx=pt[i].X;
51     if(pt[i].X>Rx) Rx=pt[i].X;
52   }
53   Ly=Ry=INF;
54   for(i=0;i<n;i++){
55     if(pt[i].X==Lx && pt[i].Y<Ly){ Ly=pt[i].Y; p=i; }
56     if(pt[i].X==Rx && pt[i].Y<Ry){ Ry=pt[i].Y; q=i; }
57   }
58   for(dn=0,i=p;i!=q;i=(i+1)%n){ qt[dn++]=pt[i]; }
59   qt[dn]=pt[q]; Ly=Ry=-INF;
60   for(i=0;i<n;i++){
61     if(pt[i].X==Lx && pt[i].Y>Ly){ Ly=pt[i].Y; p=i; }
62     if(pt[i].X==Rx && pt[i].Y>Ry){ Ry=pt[i].Y; q=i; }
63   }
64   for(un=0,i=p;i!=q;i=(i+n-1)%n){ rt[un++]=pt[i]; }
65   rt[un]=pt[q];
66}
67inline int inConvex(Pt p){
68   int L,R,M;
69   if(p.X<Lx || p.X>Rx) return 0;
70   L=0;R=dn;
71   while(L<R-1){ M=(L+R)/2;
72     if(p.X<qt[M].X) R=M; else L=M; }
73   if(tri(qt[L],qt[R],p)<0) return 0;
74   L=0;R=un;
75   while(L<R-1){ M=(L+R)/2;
76     if(p.X<rt[M].X) R=M; else L=M; }
77   if(tri(rt[L],rt[R],p)>0) return 0;
78   return 1;
79}
80int main(){
81   int n,m,i;
82   Pt p;
83   scanf("%d",&n);
84   for(i=0;i<n;i++) scanf("%lld%lld",&pt[i].X,&pt[i].Y);
85   scanf("%d",&m);
86   for(i=0;i<m;i++) scanf("%lld%lld",&qt[i].X,&qt[i].Y);
87   n=minkowskiSum(n,m);
88   for(i=0;i<n;i++) pt[i]=rt[i];
89   scanf("%d",&m);
90   for(i=0;i<m;i++) scanf("%lld%lld",&qt[i].X,&qt[i].Y);
91   n=minkowskiSum(n,m);
92   for(i=0;i<n;i++) pt[i]=rt[i];
93   initInConvex(n);
94   scanf("%d",&m);
95   for(i=0;i<m;i++){
96     scanf("%lld %lld",&p.X,&p.Y);
97     p.X*=3; p.Y*=3;
98     puts(inConvex(p)?"YES":"NO");
99   }
100}

```

## 12 Number Theory

### 12.1 Mod Sum

60beb8

```

1/**
2 * Author: Simon Lindholm
3 * Date: 2015-06-23
4 * License: CC0
5 * Source: own work
6 * Description: Sums of mod'ed arithmetic progressions.
7 *
8 * \texttt{modsum}(to, c, k, m) = \sum_{i=0}^{\texttt{modsum}(to)-1} \{ (ki+c) \% m \}.
9 * \texttt{divsum} is similar but for floored division.
10 * Time:  $\mathcal{O}(\log m)$ , with a large constant.
11 * Status: Tested for all  $|k|, |c|, to, m \leq 50$ , and on
12   kattis:aladin
13 */
14#pragma once
15typedef unsigned long long ull;
16ull sumsq(ull to) { return to / 2 * ((to-1) | 1); }
17/// ^ written in a weird way to deal with overflows correctly
18
19ull divsum(ull to, ull c, ull k, ull m) {

```

```

20 ull res = k / m * sumsq(to) + c / m * to;
21 k %= m; c %= m;
22 if (!k) return res;
23 ull to2 = (to * k + c) / m;
24 return res + (to - 1) * to2 - divsum(to2, m-1 - c, m, k);
25 }
26
27 ll modsum(ull to, ll c, ll k, ll m) {
28     c = ((c % m) + m) % m;
29     k = ((k % m) + m) % m;
30     return to * c + k * sumsq(to) - m * divsum(to, c, k, m);
31 }

```

## 12.2 Extended Lucas Theorem

3bed10

```

1 /**
2  * Author: koukanni
3  * Date: 2025
4  * License: CC0
5  * Source:
6  * Description: Computes  $C(n, m) \bmod P$  where  $P$  is not
7  * necessarily prime
8  * Functions
9  * exgcd(a, b, x, y): Extended Euclidean algorithm
10 * inv(a, p): Modular inverse of a modulo p
11 * qpow(a, b, p): Fast modular exponentiation
12 * CRT(n, a, m): Chinese Remainder Theorem
13 * calc(n, x, P): Helper function for computing factorial
14 * with prime power
15 * multilucas(n, m, x, P): Lucas theorem for prime power
16 * modulus
17 * exLucas(n, m, P): Main function - Extended Lucas theorem
18 * usage: exLucas(n, m, P) returns  $C(n, m) \bmod P$ 
19 * Time:  $O(P \log(P))$ 
20 * Status: tested
21 */
22 typedef long long ll;
23 void exgcd(ll a, ll b, ll& x, ll& y) {
24     if (!b)
25         x = 1, y = 0;
26     else
27         exgcd(b, a % b, y, x), y -= a / b * x;
28 }
29 ll inv(ll a, ll p) {
30     ll x, y;
31     exgcd(a, p, x, y);
32     return (x + p) % p;
33 }
34 ll qpow(ll a, ll b, ll p) {
35     ll r = 1;
36     for (; b; b >>= 1, a = a * a % p)
37         if (b & 1) r = r * a % p;
38     return r;
39 }
40 ll CRT(int n, ll* a, ll* m) {
41     ll M = 1, p = 0;
42     for (int i = 1; i <= n; i++) M = M * m[i];
43     for (int i = 1; i <= n; i++) {
44         ll w = M / m[i], x, y;
45         exgcd(w, m[i], x, y);
46         p = (p + a[i] * w * x % M) % M;
47     }
48     return (p % M + M) % M;
49 }
50 ll calc(ll n, ll x, ll P) {
51     if (!n) return 1;
52     ll s = 1;
53     for (ll i = 1; i <= P; i++)
54         if (i % x) s = s * i % P;
55     s = qpow(s, n / P, P);
56     for (ll i = n / P * P + 1; i <= n; i++)
57         if (i % x) s = i % P * s % P;
58     return s * calc(n / x, x, P) % P;
59 }
60 ll multilucas(ll n, ll m, ll x, ll P) {
61     if (n < m) return 0;
62     int cnt = 0;
63     for (ll i = n; i; i /= x) cnt += i / x;
64     for (ll i = m; i; i /= x) cnt -= i / x;
65     for (ll i = n - m; i; i /= x) cnt -= i / x;
66     return qpow(x, cnt, P) % P * calc(n, x, P) % P *
67         inv(calc(m, x, P), P) % P *
68         inv(calc(n - m, x, P), P) % P;
69 }
70 ll exLucas(ll n, ll m, ll P) {
71     int cnt = 0;
72     ll p[20], a[20];
73     for (ll i = 2; i * i <= P; i++) {
74         if (P % i == 0) {
75             p[++cnt] = i;

```

```

72         while (P % i == 0) p[cnt] = p[cnt] * i, P /= i;
73         a[cnt] = multilucas(n, m, i, p[cnt]);
74     }
75 }
76 if (P > 1) p[++cnt] = P, a[cnt] = multilucas(n, m, P, P);
77 return CRT(cnt, a, p);
78 }

```

## 12.3 Prime Sieve and Defactor

e2350f

```

1 // Author: Gino
2 const int maxc = 1e6 + 1;
3 vector<int> lpf;
4 vector<int> prime;
5
6 void seive() {
7     prime.clear();
8     lpf.resize(maxc, 1);
9     for (int i = 2; i < maxc; i++) {
10         if (lpf[i] == 1) {
11             lpf[i] = i;
12             prime.emplace_back(i);
13         }
14         for (auto& j : prime) {
15             if (i * j >= maxc) break;
16             lpf[i * j] = j;
17             if (j == lpf[i]) break;
18         }
19     }
20     vector<pii> fac;
21     void defactor(int u) {
22         fac.clear();
23         while (u > 1) {
24             int d = lpf[u];
25             fac.emplace_back(make_pair(d, 0));
26             while (u % d == 0) {
27                 u /= d;
28                 fac.back().second++;
29             }
30         }
31     }
32 }

```

## 12.4 Harmonic Series

7c56cc

```

1 // Author: Gino
2 //  $O(n \log n)$ 
3 for (int i = 1; i <= n; i++) {
4     for (int j = i; j <= n; j += i) {
5         //  $O(1)$  code
6     }
7 }
8
9 // PIE
10 // given array  $a[0], a[1], \dots, a[n-1]$ 
11 // calculate  $dp[x]$  = number of pairs  $(a[i], a[j])$  such that
12 //  $\gcd(a[i], a[j]) = x$  //  $(i < j)$ 
13 //
14 // idea: let  $mc(x) = \#$  of  $y$  s.t.  $x|y$ 
15 //  $f(x) = \#$  of pairs s.t.  $\gcd(a[i], a[j]) \geq x$ 
16 //  $f(x) = C(mc(x), 2)$ 
17 //  $dp[x] = f(x) - \sum(dp[y], x < y \text{ and } x|y)$ 
18 const int maxc = 1e6;
19 vector<int> cnt(maxc + 1, 0), dp(maxc + 1, 0);
20 for (int i = 0; i < n; i++)
21     cnt[a[i]]++;
22
23 for (int x = maxc; x >= 1; x--) {
24     ll cnt_mul = 0; // number of multiples of x
25     for (int y = x; y <= maxc; y += x)
26         cnt_mul += cnt[y];
27
28     dp[x] = cnt_mul * (cnt_mul - 1) / 2; // number of pairs
29     // that are divisible by x
30     for (int y = x + x; y <= maxc; y += x)
31         dp[x] -= dp[y]; // PIE: subtract all  $dp[y]$  for  $y > x$  and  $x|y$ 
32 }

```

## 12.5 Count Number of Divisors

0a5616

```

1 // Author: Gino
2 // Function: Count the number of divisors for all  $x \leq 10^6$ 
3 // using harmonic series
4 const int maxc = 1e6;
5 vector<int> facs;
6
7 void find_all_divisors() {
8     facs.clear(); facs.resize(maxc + 1, 0);
9     for (int x = 1; x <= maxc; x++) {
10         for (int y = x; y <= maxc; y += x) {
11             facs[y]++;
12         }
13     }
14 }

```

## 12.6 數論分塊

cd4e6b

```

1 // Author: Gino
2 /*

```



```

3 n = 17
4 i: 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17
5 n/i: 17 8 5 4 3 2 2 2 1 1 1 1 1 1 1 1 1
6
7           L(2)   R(2)
8
9 L(x) := left bound for n/i = x
10 R(x) := right bound for n/i = x
11
12 ===== FORMULA =====
13 >>> R = n / (n/L) <<<
14 =====
15
16 Example: L(2) = 6
17           R(2) = 17 / (17 / 6)
18           = 17 / 2
19           = 8
20 */
21 // ===== CODE =====
22
23 for (ll l = 1, r = 1, q = n; l <= n; l = r + 1) {
24     q = n/l;
25     r = n/q;
26     // Process your code here
27 }
28 // q, l, r: 17 1 1
29 // q, l, r: 8 2 2
30 // q, l, r: 5 3 3
31 // q, l, r: 4 4 4
32 // q, l, r: 3 5 5
33 // q, l, r: 2 6 8
34 // q, l, r: 1 9 17

```

## 12.7 Pollard's rho

```

1 // Author: Unknown
2 // Function: Find a non-trivial factor of a big number in
  O(n^(1/4) log^2(n))
3
4 ll find_factor(ll number) {
5     __int128 x = 2;
6     for (__int128 cycle = 1; ; cycle++) {
7         __int128 y = x;
8         for (int i = 0; i < (1<<cycle); i++) {
9             x = (x * x + 1) % number;
10            __int128 factor = __gcd(x - y, number);
11            if (factor > 1)
12                return factor;
13        }
14    }
15 }

```

```

1 # Author: Unknown
2 # Function: Find a non-trivial factor of a big number in
  O(n^(1/4) log^2(n))
3 from itertools import count
4 from math import gcd
5 from sys import stdin
6
7 for s in stdin:
8     number, x = int(s), 2
9     brk = False
10    for cycle in count(1):
11        y = x
12        if brk:
13            break
14        for i in range(1<< cycle):
15            x = (x * x + 1) % number
16            factor = gcd(x - y, number)
17            if factor > 1:
18                print(factor)
19                brk = True
20            break

```

## 12.8 Miller Rabin

686187

```

1 // Author: Unknown, Modified by Gino
2 // Function: Check if a number is a prime in O(100 *
  log^2(n))
3 // miller_rabin(): return 1 if prime, 0 otherwise
4 inline ll mul(ll x, ll y, ll mod) {
5     return (__int128)(x) * y % mod;
6 }
7 ll mypow(ll a, ll b, ll mod) {
8     ll r = 1;
9     while (b > 0) {
10        if (b & 1) r = mul(r, a, mod);
11        a = mul(a, a, mod);
12        b >>= 1;
13    }
14    return r;
15 }
16 bool witness(ll a, ll n, ll u, int t){

```

```

17 ll x = mypow(a, u, n);
18 for(int i = 0; i < t; i++) {
19     ll nx = mul(x, x, n);
20     if (nx == 1 && x != 1 && x != n-1) return true;
21     x = nx;
22 }
23 return x != 1;
24 }
25 bool miller_rabin(ll n) {
26     // if n >= 3,474,749,660,383
27     // change {2, 3, 5, 7, 11, 13} to
28     // {2, 325, 9375, 28178, 450775, 9780504, 1795265022}
29     if (n < 2) return false;
30     if (!(n & 1)) return n == 2;
31     ll u = n - 1; int t = 0;
32     while(!(u & 1)) u >>= 1, t++;
33     for (ll a : {2, 3, 5, 7, 11, 13}) {
34         if (a % n == 0) continue;
35         if (witness(a, n, u, t)) return false;
36     }
37     return true;
38 }
39 // bases that make sure no pseudoprimes flee from test
40 // if WA, replace randll(n - 1) with these bases:
41 // n < 4,759,123,141      3 : 2, 7, 61
42 // n < 1,122,004,669,633  4 : 2, 13, 23, 1662803
43 // n < 3,474,749,660,383  6 : pimes <= 13
44 // n < 2^64              7 :
45 // 2, 325, 9375, 28178, 450775, 9780504, 1795265022

```

## 12.9 Discrete Log

a04e88

```

1 // Author: ckiseki
2 // find x^? = y (mod M)
3 // O(sqrt(M))
4 template<typename Int>
5 Int BSGS(Int x, Int y, Int M) {
6     // x^? |equiv y (mod M)
7     Int t = 1, c = 0, g = 1;
8     for (Int M_ = M; M_ > 0; M_ >>= 1) g = g * x % M;
9     for (g = gcd(g, M); t % g != 0; ++c) {
10        if (t == y) return c;
11        t = t * x % M;
12    }
13    if (y % g != 0) return -1;
14    t /= g, y /= g, M /= g;
15    Int h = 0, gs = 1;
16    for (; h * h < M; ++h) gs = gs * x % M;
17    unordered_map<Int, Int> bs;
18    for (Int s = 0; s < h; bs[s] = ++s) y = y * x % M;
19    for (Int s = 0; s < M; s += h) {
20        t = t * gs % M;
21        if (bs.count(t)) return c + s + h - bs[t];
22    }
23    return -1;
24 }

```

## 12.10 Discrete Sqrt

3f16e1

```

1 // Author: ckiseki
2 // find solution for x^2 = n (mod P)
3 // O(log^2 P)
4 int get_root(int n, int P) { // ensure 0 <= n < p
5     if (P == 2 or n == 0) return n;
6     auto check = [&](ll d) {
7         return modpow(int(x), (P - 1) / 2, P);
8     };
9     if (check(n) != 1) return -1;
10    mt19937 rnd(7122); ll d = 1, w;
11    while (check(w = (z * z - n + P) % P) != P - 1)
12        z = rnd() % P;
13    const auto M = [P, w](auto &u, auto &v) {
14        auto [a, b] = u; auto [c, d] = v;
15        return make_pair((a * c + b * d % P * w) % P,
16            (a * d + b * c) % P);
17    };
18    pair<ll, ll> r(1, 0), e(2, 1);
19    for (int q = (P + 1) / 2; q >>= 1; e = M(e, e))
20        if (q & 1) r = M(r, e);
21    return int(r.first); // sqrt(n) mod P where P is prime

```

## 12.11 Fast Power

Note:  $a^n \equiv a^{(n \bmod (p-1))} \pmod{p}$ 

## 12.12 Extend GCD

5eb7fc

```

1 // Author: Gino
2 // [Usage]
3 // bezout(a, b, c):
4 //     find solution to ax + by = c
5 //     return {-LINF, -LINF} if no solution
6 // inv(a, p):
7 //     find modulo inverse of a under p

```

```

8//      return -1 if not exist
9// CRT(vector<ll>& a, vector<ll>& m)
10//      find a solution pair (x, mod) satisfies all x = a[i]
11//      (mod m[i])
12//      return {-LINF, -LINF} if no solution
13const ll LINF = 4e18;
14typedef pair<ll, ll> pll;
15template<typename T1, typename T2>
16T1 chmod(T1 a, T2 m) {
17    return (a % m + m) % m;
18}
19
20ll GCD;
21pll extgcd(ll a, ll b) {
22    if (b == 0) {
23        GCD = a;
24        return pll{1, 0};
25    }
26    pll ans = extgcd(b, a % b);
27    return pll{ans.second, ans.first - a/b * ans.second};
28}
29pll bezout(ll a, ll b, ll c) {
30    bool negx = (a < 0), negy = (b < 0);
31    pll ans = extgcd(abs(a), abs(b));
32    if (c % GCD != 0) return pll{-LINF, -LINF};
33    return pll{ans.first * c/GCD * (negx ? -1 : 1),
34              ans.second * c/GCD * (negy ? -1 : 1)};
35}
36ll inv(ll a, ll p) {
37    if (p == 1) return -1;
38    pll ans = bezout(a % p, -p, 1);
39    if (ans == pll{-LINF, -LINF}) return -1;
40    return chmod(ans.first, p);
41}
42pll CRT(vector<ll>& a, vector<ll>& m) {
43    for (int i = 0; i < (int)a.size(); i++)
44        a[i] = chmod(a[i], m[i]);
45
46    ll x = a[0], mod = m[0];
47    for (int i = 1; i < (int)a.size(); i++) {
48        pll sol = bezout(mod, m[i], a[i] - x);
49        if (sol.first == -LINF) return pll{-LINF, -LINF};
50
51        // prevent long long overflow
52        ll p = chmod(sol.first, m[i] / GCD);
53        ll lcm = mod / GCD * m[i];
54        x = chmod((__int128)p * mod + x, lcm);
55        mod = lcm;
56    }
57    return pll{x, mod};
58}

```

### 12.13 Mu + Phi

ab4a91

```

1// Author: Gino
2const int maxn = 1e6 + 5;
3ll f[maxn];
4vector<int> lpf, prime;
5void build() {
6    lpf.clear(); lpf.resize(maxn, 1);
7    prime.clear();
8    f[1] = ...; /* mu[1] = 1, phi[1] = 1 */
9    for (int i = 2; i < maxn; i++) {
10        if (lpf[i] == 1) {
11            lpf[i] = i; prime.emplace_back(i);
12            f[i] = ...; /* mu[i] = 1, phi[i] = i-1 */
13        }
14        for (auto& j : prime) {
15            if (i*j >= maxn) break;
16            lpf[i*j] = j;
17            if (i % j == 0) f[i*j] = ...; /* 0, phi[i]*j */
18            else f[i*j] = ...; /* -mu[i], phi[i]*phi[j] */
19            if (j >= lpf[i]) break;
20        }
21    }
22}

```

### 12.14 Other Formulas

- Pisano Period: 任何線性遞迴（比如費氏數列）模任何一個數字  $M$  都會循環，找循環節  $\pi(M)$  先質因數分解  $M = \prod p_i^{e_i}$ ，然後  $\pi(M) = \text{lcm}(\pi(p_i^{e_i}))$ ，
- Inversion:  $aa^{-1} \equiv 1(\text{mod } m)$ .  $a^{-1}$  exists iff  $\text{gcd}(a, m) = 1$ .
- Linear inversion:  $a^{-1} \equiv (m - \lfloor \frac{m}{a} \rfloor) \times (m \text{ mod } a)^{-1}(\text{mod } m)$
- Fermat's little theorem:  $a^p \equiv a(\text{mod } p)$  if  $p$  is prime.
- Euler function:  $\varphi(n) = n \prod_{p|n} \frac{p-1}{p}$

- Euler theorem:  $a^{\varphi(n)} \equiv 1(\text{mod } n)$  if  $\text{gcd}(a, n) = 1$ . If  $a, n$  are not coprime: 質因數分解  $n = \prod p_i^{e_i}$ ，對每個  $p_i^{e_i}$  分開看他們跟  $a$  是否互質（互質：Fermat / 不互質：夠大的指數會直接削成 0），最後用 CRT 合併。
- Extended Euclidean algorithm:  $ax + by = \text{gcd}(a, b) = \text{gcd}(b, a \text{ mod } b) = \text{gcd}(b, a - \lfloor \frac{a}{b} \rfloor b) = bx_1 + (a - \lfloor \frac{a}{b} \rfloor b)y_1 = ay_1 + b(x_1 - \lfloor \frac{a}{b} \rfloor y_1)$
- Divisor function:  $\sigma_x(n) = \sum_{d|n} d^x$ .  $n = \prod_{i=1}^r p_i^{a_i}$ .  
 $\sigma_x(n) = \prod_{i=1}^r \frac{p_i^{(a_i+1)x} - 1}{p_i^x - 1}$  if  $x \neq 0$ .  $\sigma_0(n) = \prod_{i=1}^r (a_i + 1)$ .
- Chinese remainder theorem (Coprime Moduli):  
 $x \equiv a_i(\text{mod } m_i)$ .  
 $M = \prod m_i$ .  $M_i = \frac{M}{m_i}$ .  $t_i = M_i^{-1}$ .  
 $x = kM + \sum a_i t_i M_i, k \in \mathbb{Z}$ .
- Chinese remainder theorem:  
 $x \equiv a_1(\text{mod } m_1), x \equiv a_2(\text{mod } m_2) \Rightarrow x = m_1 p + a_1 = m_2 q + a_2 \Rightarrow m_1 p - m_2 q = a_2 - a_1$   
Solve for  $(p, q)$  using ExtGCD.  
 $x \equiv m_1 p + a_1 \equiv m_2 q + a_2(\text{mod } \text{lcm}(m_1, m_2))$
- Avoiding Overflow:  $ca \text{ mod } cb = c(a \text{ mod } b)$
- Dirichlet Convolution:  $(f * g)(n) = \sum_{d|n} f(d)g(\frac{n}{d})$
- Important Multiplicative Functions + Properties:
  - $\varepsilon(n) = [n = 1]$
  - $1(n) = 1$
  - $\text{id}(n) = n$
  - $\mu(n) = 0$  if  $n$  has squared prime factor
  - $\mu(n) = (-1)^k$  if  $n = p_1 p_2 \dots p_k$
  - $\varepsilon = \mu * 1$
  - $\varphi = \mu * \text{id}$
  - $[n = 1] = \sum_{d|n} \mu(d)$
  - $[\text{gcd} = 1] = \sum_{d|\text{gcd}} \mu(d)$
- Möbius inversion:  $f = g * 1 \Leftrightarrow g = f * \mu$

### 12.15 Polynomial

57318b

```

1// Author: Gino
2// Preparation: first set_mod(mod, g), then init_ntt()
3// everytime you change modulus, you have to call
4// init_ntt() again
5// [Usage]
6// polynomial: vector<ll> a, b
7// negation: -a
8// add/subtract: a += b, a -= b
9// convolution: a *= b
10// in-place modulo: mod(a, b)
11// in-place inversion under mod x^N: inv(ia, N)
12
13const int maxk = 20;
14const int maxn = 1<<maxk;
15
16using u64 = unsigned long long;
17using u128 = __uint128_t;
18
19int g;
20u64 MOD;
21u64 BARRETT_IM; // [ 2^64 / MOD ]
22
23inline void set_mod(u64 m, int _g) {
24    g = _g;
25    MOD = m;
26    BARRETT_IM = (u128(1) << 64) / m;
27}
28
29inline u64 chmod(u128 x) {
30    u64 q = (u64)((x * BARRETT_IM) >> 64);
31    u64 r = (u64)(x - (u128)q * MOD);
32    if (r >= MOD) r -= MOD;
33    return r;
34}
35
36inline u64 mmul(u64 a, u64 b) {
37    return chmod((u128)a * b);
38}
39
40ll pw(ll a, ll n) {
41    ll ret = 1;
42    while (n > 0) {
43        if (n & 1) ret = mmul(ret, a);
44        a = mmul(a, a);
45        n >>= 1;
46    }
47    return ret;
48}
49
50vector<ll> X, iX;
51vector<int> rev;

```

```

50 void init_ntt() {
51     X.assign(maxn, 1); // x1 = g^((p-1)/n)
52     iX.assign(maxn, 1);
53
54     ll u = pw(g, (MOD-1)/maxn);
55     ll iu = pw(u, MOD-2);
56     for (int i = 1; i < maxn; i++) {
57         X[i] = mmul(X[i-1], u);
58         iX[i] = mmul(iX[i-1], iu);
59     }
60
61     if ((int)rev.size() == maxn) return;
62     rev.assign(maxn, 0);
63     for (int i = 1, hb = -1; i < maxn; i++) {
64         if (!(i & (i-1))) hb++;
65         rev[i] = rev[i ^ (1<<hb)] | (1<<(maxk-hb-1));
66     }
67 template<typename T>
68 void NTT(vector<T>& a, bool inv=false) {
69     int _n = (int)a.size();
70     int k = __lg(_n) + ((1<<__lg(_n)) != _n);
71     int n = 1<<k;
72     a.resize(n, 0);
73
74     short shift = maxk-k;
75     for (int i = 0; i < n; i++)
76         if (i > (rev[i]>>shift))
77             swap(a[i], a[rev[i]>>shift]);
78     for (int len = 2, half = 1, div = maxn>>1; len <= n;
79         len<<=1, half<<=1, div>>=1) {
80         for (int i = 0; i < n; i += len) {
81             for (int j = 0; j < half; j++) {
82                 T u = a[i+j];
83                 T v = mmul(a[i+j+half], (inv ? iX[j*div] :
84                     X[j*div]));
85                 a[i+j] = (u+v >= MOD ? u+v-MOD : u+v);
86                 a[i+j+half] = (u-v < 0 ? u-v+MOD : u-v);
87             } }
88         if (inv) {
89             T dn = pw(n, MOD-2);
90             for (auto& x : a) {
91                 x = mmul(x, dn);
92             } }
93     }
94 template<typename T>
95 inline void shrink(vector<T>& a) {
96     int cnt = (int)a.size();
97     for (; cnt > 0; cnt--) if (a[cnt-1]) break;
98     a.resize(max(cnt, 1));
99 }
100 template<typename T>
101 vector<T>& operator*=(vector<T>& a, vector<T> b) {
102     int na = (int)a.size();
103     int nb = (int)b.size();
104     a.resize(na + nb - 1, 0);
105     b.resize(na + nb - 1, 0);
106     NTT(a); NTT(b);
107     for (int i = 0; i < (int)a.size(); i++)
108         a[i] = mmul(a[i], b[i]);
109     NTT(a, true);
110     shrink(a);
111     return a;
112 }
113 inline ll crt(ll a0, ll a1, ll m1, ll m2, ll
114     inv_m1_mod_m2){
115     // x ≡ a0 (mod m1), x ≡ a1 (mod m2)
116     // t = (a1 - a0) * inv(m1) mod m2
117     // x = a0 + t * m1 (mod m1*m2)
118     ll t = chmod(a1 - a0);
119     if (t < 0) t += m2;
120     t = (ll)((__int128)t * inv_m1_mod_m2 % m2);
121     return a0 + (ll)((__int128)t * m1);
122 }
123 void mul_crt() {
124     // a copy to a1, a2 | b copy to b1, b2
125     ll M1 = 998244353, M2 = 1004535809;
126     g = 3; set_mod(M1); init_ntt(); a1 *= b1;
127     g = 3, set_mod(M2); init_ntt(); a2 *= b2;
128
129     ll inv_m1_mod_m2 = pw(M1, M2 - 2);
130     for (int i = 2; i <= 2 * k; i++)
131         cout << crt(a1[i], a2[i], M1, M2, inv_m1_mod_m2)
132         << ' ';
133     cout << endl;
134 }
135 /* P = r*2^k + 1
136 P

```

```

135 998244353      119 23 3
136 1004535809    479 21 3
137
138 P              r   k   g
139 3              1   1   2
140 5              1   2   2
141 17             1   4   3
142 97            3   5   5
143 193           3   6   5
144 257           1   8   3
145 7681          15  9  17
146 12289         3  12  11
147 40961         5  13  3
148 65537         1  16  3
149 786433        3  18  10
150 5767169       11  19  3
151 7340033       7  20  3
152 23068673      11  21  3
153 104857601     25  22  3
154 167772161     5  25  3
155 469762049     7  26  3
156 1004535809   479 21  3
157 2013265921   15  27  31
158 2281701377   17  27  3
159 3221225473   3  30  5
160 75161927681  35  31  3
161 77309411329  9  33  7
162 206158430209 3  36  22
163 2061584302081 15 37  7
164 2748779069441 5  39  3
165 6597069766657 3  41  5
166 39582418599937 9  42  5
167 79164837199873 9  43  5
168 263882790666241 15 44  7
169 1231453023109121 35 45  3
170 1337006139375617 19 46  3
171 3799912185593857 27 47  5
172 4222124650659841 15 48  19
173 7881299347898369 7  50  6
174 31525197391593473 7  52  3
175 180143985094819841 5  55  6
176 1945555039024054273 27 56  5
177 4179340454199820289 29 57  3
178 9097271247288401921 505 54  6 */

```

## 12.16 Counting Primes

1662d0

```

1 // prime_count - #primes in [1..n] (O(n^{2/3}) time,
2 // O(sqrt(n)) memory)
3 using u64 = unsigned long long;
4 static inline u64 prime_count(u64 n){
5     if(n<=1) return 0;
6     int v = (int)floor(sqrt((long double)n));
7     int s = (v+1)>>1, pc = 0;
8     vector<int> smalls(s), roughs(s), skip(v+1);
9     vector<long long> larges(s);
10
11     for(int i=0;i<s;++i){
12         smalls[i]=i;
13         roughs[i]=2*i+1;
14         larges[i]=(long long)((n/roughs[i]-1)>>1);
15     }
16
17     for(int p=3;p<=v;p+=2) if(!skip[p]){
18         int q = p*p;
19         if(1LL*q*q > (long long)n) break;
20         skip[p]=1;
21         for(int i=q;i<=v;i+=2*p) skip[i]=1;
22
23         int ns=0;
24         for(int k=0;k<s;++k){
25             int i = roughs[k];
26             if(skip[i]) continue;
27             u64 d = (u64)i * (u64)p;
28             long long sub = (d <= (u64)v)
29                 ? larges[smalls[(int)(d>>1)] - pc]
30                 : smalls[(int)((n/d - 1) >> 1)];
31             larges[ns] = larges[k] - sub + pc;
32             roughs[ns++] = i;
33         }
34         s = ns;
35         for(int i=(v-1)>>1, j=((v/p)-1)|1; j>=p; j-=2){
36             int c = smalls[j>>1] - pc;
37             for(int e=(j*p)>>1; i>=e; --i) smalls[i] -= c;
38         }
39         ++pc;
40     }
41     larges[0] += 1LL*(s + 2*(pc-1))*(s-1) >> 1;

```

```

43 for(int k=1;k<s;++k) larges[0] -= larges[k];
44
45 for(int l=1;l<s;++l){
46     int q = roughs[l];
47     u64 m = n / (u64)q;
48     long long t = 0;
49     int e = smalls[(int)((m/q - 1) >> 1)] - pc;
50     if(e < l+1) break;
51     for(int k=l+1;k<=e;++k) t += smalls[(int)((m/
(u64)roughs[k] - 1) >> 1)];
52     larges[0] += t - 1LL*(e - l)*(pc + l - 1);
53 }
54 return (u64)(larges[0] + 1);
55 }

```

## 12.17 Linear Sieve for Other Number Theoretic Functions

```

1// linear_sieve(n, primes, lp, phi, mu, d, sigma)
2// Outputs over the index range 0..n (n >= 1):
3// primes : all primes in [2..n], increasing.
4// lp      : lowest prime factor; lp[1]=0, lp[x] is the
5//           smallest prime dividing x.
6// phi     : Euler totient, phi[x] = |{1<=k<=x :
7//           gcd(k,x)=1}|. Multiplicative.
8// mu      : Möbius; mu[1]=1, mu[x]=0 if x has a squared
9//           prime factor, else (-1)^(#distinct primes).
10// d       : number of divisors; if x=∏ p_i^{e_i}, then
11//           d[x]=∏(e_i+1). Multiplicative.
12// sigma   : sum of divisors; if x=∏ p_i^{e_i}, then
13//           sigma[x]=∏(1+p_i+...+p_i^{e_i}). (use ll)
14//
15// Complexity: O(n) time, O(n) memory.
16// Notes: Arrays are resized inside; primes is cleared and
17//         reserved. sigma uses ll to avoid 32-bit overflow.
18
19 static inline void linear_sieve(
20     int n,
21     std::vector<int> &primes,
22     std::vector<int> &lp,
23     std::vector<int> &phi,
24     std::vector<int> &mu,
25     std::vector<int> &d,
26     std::vector<ll> &sigma
27 ) {
28     lp.assign(n + 1, 0); phi.assign(n + 1, 0); mu.assign(n +
29     1, 0); d.assign(n + 1, 0); sigma.assign(n + 1, 0);
30     primes.clear(); primes.reserve(n > 1 ? n / 10 : 0);
31     std::vector<int> cnt(n + 1, 0), core(n + 1, 1);
32     std::vector<ll> p_pow(n + 1, 1), sum_p(n + 1, 1);
33     phi[1] = mu[1] = d[1] = sigma[1] = 1;
34
35     for (int i = 2; i <= n; ++i) {
36         if (!lp[i]) {
37             lp[i] = i; primes.push_back(i);
38             phi[i] = i - 1; mu[i] = -1; d[i] = 2;
39             cnt[i] = 1; p_pow[i] = i; core[i] = 1;
40             sum_p[i] = 1 + (ll)i; sigma[i] = sum_p[i];
41         }
42         for (int p : primes) {
43             long long ip = 1LL * i * p;
44             if (ip > n) break;
45             lp[ip] = p;
46             if (p == lp[i]) {
47                 cnt[ip] = cnt[i] + 1; p_pow[ip] = p_pow[i] * p;
48                 core[ip] = core[i];
49                 sum_p[ip] = sum_p[i] + p_pow[ip];
50                 phi[ip] = phi[i] * p; mu[ip] = 0;
51                 d[ip] = d[core[ip]] * (cnt[ip] + 1);
52                 sigma[ip] = sigma[core[ip]] * sum_p[ip];
53                 break; // critical for linear complexity
54             } else {
55                 cnt[ip] = 1; p_pow[ip] = p; core[ip] = i;
56                 sum_p[ip] = 1 + (ll)p;
57                 phi[ip] = phi[i] * (p - 1); mu[ip] = -mu[i];
58                 d[ip] = d[i] * 2;
59                 sigma[ip] = sigma[i] * sum_p[ip];
60             }
61         }
62     }
63 }
64
65 // Optional helper: factorize x in O(log x) using lp
66 // (requires x in [2..n])
67 static inline std::vector<std::pair<int,int>>
68 factorize(int x, const std::vector<int> &lp) {
69     std::vector<std::pair<int,int>> res;
70     while (x > 1) {
71         int p = lp[x], e = 0;
72         do { x /= p; ++e; } while (x % p == 0);
73         res.push_back({p, e});
74     }
75 }

```

```

65 return res;
66 }

```

## 13 Numerical

### 13.1 Polynomials and recurrences

```

1/* Author: David Rydh, Per Austrin
2* Description: */
3#pragma once
4struct Poly {
5    vector<double> a;
6    double operator()(double x) const {
7        double val = 0;
8        for (int i = sz(a); i--;) (val *= x) += a[i];
9        return val;
10    }
11    void diff() {
12        rep(i, 1, sz(a)) a[i-1] = i*a[i];
13        a.pop_back();
14    }
15    void divroot(double x0) {
16        double b = a.back(), c; a.back() = 0;
17        for(int i=sz(a)-1; i--;) c = a[i], a[i] = a[i+1]*x0+b,
18        b=c;
19        a.pop_back();
20    };
21
22    /* Author: Per Austrin (License: CC0)
23    * Description: Finds the real roots to a polynomial.
24    * Usage: polyRoots({{2,-3,1}}, -1e9, 1e9) // solve x^2-3x+2 =
25    *        0
26    * Time: O(n^2 \log(1/epsilon)) */
27    #include "Polynomial.h"
28    vector<double> polyRoots(Poly p, double xmin, double xmax) {
29        if (sz(p.a) == 2) { return {-p.a[0]/p.a[1]}; }
30        vector<double> ret;
31        Poly der = p;
32        der.diff();
33        auto dr = polyRoots(der, xmin, xmax);
34        dr.push_back(xmin-1);
35        dr.push_back(xmax+1);
36        sort(all(dr));
37        rep(i, 0, sz(dr)-1) {
38            double l = dr[i], h = dr[i+1];
39            bool sign = p(l) > 0;
40            if (sign ^ (p(h) > 0)) {
41                rep(it, 0, 60) { // while (h - l > 1e-8)
42                    double m = (l + h) / 2, f = p(m);
43                    if ((f <= 0) ^ sign) l = m;
44                    else h = m;
45                }
46                ret.push_back((l + h) / 2);
47            }
48        }
49        return ret;
50    }
51
52    /* Author: Simon Lindholm (License: CC0)
53    * Description: Given $n$ points $(x[i], y[i])$, computes an
54    *               $n-1$-degree polynomial $p$ such that
55    *               $p$ passes through them: $p(x) = a[0]*x^0 + \dots +
56    *               a[n-1]*x^{n-1}$.
57    *               For numerical precision, pick $x[k] = c*\cos(k/(n-1)*\pi)$,
58    *               $k=0 \dots n-1$.
59    * Time: O(n^2) */
60    #pragma once
61    typedef vector<double> vd;
62    vd interpolate(vd x, vd y, int n) {
63        vd res(n), temp(n);
64        rep(k, 0, n-1) rep(i, k+1, n)
65            y[i] = (y[i] - y[k]) / (x[i] - x[k]);
66        double last = 0; temp[0] = 1;
67        rep(k, 0, n) rep(i, 0, n) {
68            res[i] += y[k] * temp[i];
69            swap(last, temp[i]);
70            temp[i] -= last * x[k];
71        }
72        return res;
73    }
74
75    /* Author: koukanni (License: CC0)
76    * Description: Lagrange interpolation for polynomial
77    *               evaluation at integer points.
78    *               Given polynomial values $f[0], f[1], \dots, f[n-1]$, computes
79    *               $f(c)$.
80    * Time: O(n)
81    * Status: tested */
82    Mint lagrange_iota(const vector<Mint> &f, const int c) {
83        const int n = int(size(f));
84        if (c < n) {

```



```

9     return f[c];
10 }
11 vector<Mint> fac(n), ifac(n);
12 fac[0] = 1;
13 for (int i = 1; i < n; i++) fac[i] = fac[i - 1] * i;
14 ifac[n - 1] = fac[n - 1].inv();
15 for (int i = n - 1; i >= 1; i--) ifac[i - 1] = ifac[i] *
i;
16 vector<Mint> g(n);
17 for (int i = 0; i < n; i++) {
18     g[i] = f[i] * ifac[i] * ifac[n - 1 - i];
19     if ((n - 1 - i) & 1) g[i] *= -1;
20 }
21 vector<Mint> lp(n + 1), rp(n + 1);
22 lp[0] = rp[n] = 1;
23 for (int i = 0; i < n; i++) lp[i + 1] = lp[i] * Mint(c -
i);
24 for (int i = n - 1; i >= 0; i--) rp[i] = rp[i + 1] *
Mint(c - i);
25 Mint ans = 0;
26 for (int i = 0; i < n; i++) ans += g[i] * lp[i] * rp[i +
1];
27 return ans;
28 }

/* Author: Lucian Bicsi (License: CC0)
* Description: Recovers any $n$-order linear recurrence
relation from the first
* $2n$ terms of the recurrence.
* Useful for guessing linear recurrences after brute-forcing
the first terms.
* Should work on any field, but numerical stability for
floats is not guaranteed.
* Output will have size $\le n$.
* Usage: berlekampMassey({0, 1, 1, 3, 5, 11}) // {1, 2}
* Time: $O(N^2)$, Status: brute-force-tested mod 5 for $n \le 5$
and all $s$ */
#include <number-theory/ModPow.h>
vector<ll> berlekampMassey(vector<ll> s) {
11     int n = sz(s), L = 0, m = 0;
12     vector<ll> C(n), B(n), T;
13     C[0] = B[0] = 1;
14     ll b = 1;
15     rep(i, 0, n) { ++m;
16         ll d = s[i] % mod;
17         rep(j, 1, L + 1) d = (d + C[j] * s[i - j]) % mod;
18         if (!d) continue;
19         T = C; ll coef = d * modpow(b, mod - 2) % mod;
20         rep(j, m, n) C[j] = (C[j] - coef * B[j - m]) % mod;
21         if (2 * L > i) continue;
22         L = i + 1 - L; B = T; b = d; m = 0;
23     }
24     // assert(2 * L + 30 <= n);
25     C.resize(L + 1); C.erase(C.begin());
26     for (ll& x : C) x = (mod - x) % mod;
27     return C;
28 }

/* Author: Lucian Bicsi (License: CC0)
* Description: Generates the $k$'th term of an $n$-order
* linear recurrence $S[i] = \sum_j S[i-j]tr[j]$,
* given $S[0 \dots l_1]$ and $tr[0 \dots n-1]$.
* Faster than matrix multiplication.
* Useful together with Berlekamp-Massey.
* Usage: linearRec({0, 1}, {1, 1}, k) // $k$'th Fibonacci
number
* Time: $O(n^2 \log k)$
* Status: brute-force-tested mod 5 for $n \le 5$ */
const ll mod = 5; //** exclude-line */
typedef vector<ll> Poly;
ll linearRec(Poly S, Poly tr, ll k) {
13     int n = sz(tr);
14     auto combine = [&](Poly a, Poly b) {
15         Poly res(n * 2 + 1);
16         rep(i, 0, n + 1) rep(j, 0, n + 1)
17             res[i + j] = (res[i + j] + a[i] * b[j]) % mod;
18         for (int i = 2 * n; i > n; --i) rep(j, 0, n)
19             res[i - 1 - j] = (res[i - 1 - j] + res[i] * tr[j]) %
mod;
20         res.resize(n + 1);
21         return res;
22     };
23     Poly pol(n + 1, e(pol));
24     pol[0] = e[1] = 1;
25     for (++k; k; k /= 2) {
26         if (k % 2) pol = combine(pol, e);
27         e = combine(e, e);
28     }
29     ll res = 0;
30     rep(i, 0, n) res = (res + pol[i + 1] * S[i]) % mod;

```

```

31     return res;
32 }

```

## 13.2 Optimization

```

/* Author: Ulf Lundstrom (License: CC0)
* Description: Finds the argument minimizing the function
$f$ in the interval $[a,b]$
* assuming $f$ is unimodal on the interval, i.e. has only
one local minimum and no local
* maximum. The maximum error in the result is $eps$. Works
equally well for maximization
* with a small change in the code. See TernarySearch.h in
the Various chapter for a
* discrete version.
* Usage: double func(double x) { return 4+x+.3*x*x; }
* double xmin = gss(-1000,1000,func);
* Time: $O(\log((b-a) / \epsilon))$, Status: tested */
// It is important for $r$ to be precise, otherwise we don't
necessarily maintain the inequality $a < x_1 < x_2 < b$.
double gss(double a, double b, double (*f)(double)) {
12     double r = (sqrt(5)-1)/2, eps = 1e-7;
13     double x1 = b - r*(b-a), x2 = a + r*(b-a);
14     double f1 = f(x1), f2 = f(x2);
15     while (b-a > eps)
16         if (f1 < f2) { //change to > to find maximum
17             b = x2; x2 = x1; f2 = f1;
18             x1 = b - r*(b-a); f1 = f(x1);
19         } else {
20             a = x1; x1 = x2; f1 = f2;
21             x2 = a + r*(b-a); f2 = f(x2);
22         }
23     return a;
24 }

/* Author: Simon Lindholm (License: CC0)
* Description: Poor man's optimization for unimodal
functions.
* Status: used with great success */
typedef array<double, 2> P;
template<class F> pair<double, P> hillClimb(P start, F f) {
6     pair<double, P> cur(f(start), start);
7     for (double jmp = 1e9; jmp > 1e-20; jmp /= 2) {
8         rep(j, 0, 100) rep(dx, -1, 2) rep(dy, -1, 2) {
9             P p = cur.second;
10            p[0] += dx*jmp;
11            p[1] += dy*jmp;
12            cur = min(cur, make_pair(f(p), p));
13        }
14    }
15    return cur;
16 }

/* Author: Simon Lindholm (License: CC0)
* Description: Simple integration of a function over an
interval using
* Simpson's rule. The error should be proportional to $h^4$,
although in
* practice you will want to verify that the result is stable
to desired
* precision when epsilon changes.
* Status: mostly untested */
template<class F>
double quad(double a, double b, F f, const int n = 1000) {
9     double h = (b - a) / 2 / n, v = f(a) + f(b);
10    rep(i, 1, n * 2)
11        v += f(a + i * h) * (i & 1 ? 4 : 2);
12    return v * h / 3;
13 }

/* Author: Simon Lindholm (License: CC0)
* Description: Fast integration using an adaptive Simpson's
rule.
* Usage:
* double sphereVolume = quad(-1, 1, [](double x) {
5* return quad(-1, 1, [](double y) {
6* return quad(-1, 1, [](double z) {
7* return x*x + y*y + z*z < 1; });});});
* Status: mostly untested */
typedef double d;
#define S(a,b) (f(a) + 4*f((a+b) / 2) + f(b)) * (b-a) / 6
template<class F>
d rec(F& f, d a, d b, d eps, d S) {
13     d c = (a + b) / 2;
14     d S1 = S(a, c), S2 = S(c, b), T = S1 + S2;
15     if (abs(T - S) <= 15 * eps || b - a < 1e-10)
16         return T + (T - S) / 15;
17     return rec(f, a, c, eps / 2, S1) + rec(f, c, b, eps / 2,
S2);
18 }
template<class F>
d quad(d a, d b, F f, d eps = 1e-8) {

```

```

21 return rec(f, a, b, eps, S(a, b));
22 }

/* Author: Stanford (License: MIT)
2 * Description: Solves a general linear maximization problem:
   maximize $c^T x$ subject to $Ax \le b$, $x \ge 0$.
3 * Returns -inf if there is no solution, inf if there are
   arbitrarily good solutions, or the maximum value of $c^T x$
   otherwise.
4 * The input vector is set to an optimal $x$ (or in the
   unbounded case, an arbitrary solution fulfilling the
   constraints).
5 * Numerical stability is not guaranteed. For better
   performance, define variables such that $x = 0$ is viable.
6 * Usage:
7 * vvd A = {{1,-1}, {-1,1}, {-1,-2}};
8 * vd b = {1,1,-4}, c = {-1,-1}, x;
9 * T val = LPSolver(A, b, c).solve(x);
10 * Time: $O(NM * \#pivots)$, where a pivot may be e.g. an edge
   relaxation. $O(2^n)$ in the general case.
11 * Status: seems to work? */
12 typedef double T; // long double, Rational, double +
   mod<P>...
13 typedef vector<T> vd;
14 typedef vector<vd> vvd;
15 const T eps = 1e-8, inf = 1/.0;
16 #define MP make_pair
17 #define ltj(X) if(s == -1 || MP(X[j],N[j]) < MP(X[s],N[s]))
   s=j
18 struct LPSolver {
19     int m, n; vi N, B; vvd D;
20     LPSolver(const vvd& A, const vd& b, const vd& c) :
21         m(sz(b)), n(sz(c)), N(n+1), B(m), D(m+2, vd(n+2)) {
22         rep(i,0,m) rep(j,0,n) D[i][j] = A[i][j];
23         rep(i,0,m) { B[i] = n+i; D[i][n] = -1; D[i][n+1] =
24             b[i]; }
25         rep(j,0,n) { N[j] = j; D[m][j] = -c[j]; }
26         N[n] = -1; D[m+1][n] = 1;
27     }
28     void pivot(int r, int s) {
29         T *a = D[r].data(), inv = 1 / a[s];
30         rep(i,0,m+2) if (i != r && abs(D[i][s]) > eps) {
31             T *b = D[i].data(), inv2 = b[s] * inv;
32             rep(j,0,n+2) b[j] -= a[j] * inv2;
33             b[s] = a[s] * inv2;
34         }
35         rep(j,0,n+2) if (j != s) D[r][j] *= inv;
36         rep(i,0,m+2) if (i != r) D[i][s] *= -inv;
37         D[r][s] = inv;
38         swap(B[r], N[s]);
39     }
40     bool simplex(int phase) {
41         int x = m + phase - 1;
42         for (;;) {
43             int s = -1;
44             rep(j,0,n+1) if (N[j] != -phase) ltj(D[X]);
45             if (D[X][s] >= -eps) return true;
46             int r = -1;
47             rep(i,0,m) {
48                 if (D[i][s] <= eps) continue;
49                 if (r == -1 || MP(D[i][n+1] / D[i][s], B[i])
50                     < MP(D[r][n+1] / D[r][s], B[r])) r = i;
51             }
52             if (r == -1) return false;
53             pivot(r, s);
54         }
55     }
56     T solve(vd &x) {
57         int r = 0;
58         rep(i,1,m) if (D[i][n+1] < D[r][n+1]) r = i;
59         if (D[r][n+1] < -eps) {
60             pivot(r, n);
61             if (!simplex(2) || D[m+1][n+1] < -eps) return -inf;
62             rep(i,0,m) if (B[i] == -1) {
63                 int s = 0;
64                 rep(j,1,n+1) ltj(D[i]);
65                 pivot(i, s);
66             }
67         }
68         bool ok = simplex(1); x = vd(n);
69         rep(i,0,m) if (B[i] < n) x[B[i]] = D[i][n+1];
70         return ok ? D[m][n+1] : inf;
71     }
};

```

### 13.3 Matrices

```

/* Author: Simon Lindholm (License: CC0)
2 * Description: Calculates determinant of a matrix. Destroys
   the matrix.
3 * Time: $O(N^3)$, Status: somewhat tested */
4 double det(vector<vector<double>>& a) {

```

```

5     int n = sz(a); double res = 1;
6     rep(i,0,n) {
7         int b = i;
8         rep(j,i+1,n) if (fabs(a[j][i]) > fabs(a[b][i])) b = j;
9         if (i != b) swap(a[i], a[b]), res *= -1;
10        res *= a[i][i];
11        if (res == 0) return 0;
12        rep(j,i+1,n) {
13            double v = a[j][i] / a[i][i];
14            if (v != 0) rep(k,i+1,n) a[j][k] -= v * a[i][k];
15        }
16    }
17    return res;
18 }

/* Author: Unknown
2 * Description: Calculates determinant using modular
   arithmetics.
3 * Modulos can also be removed to get a pure-integer version.
4 * Time: $O(N^3)$
5 * Status: brute-force-tested for $N \le 3$, mod $\le 7$ */
6 const ll mod = 12345;
7 ll det(vector<vector<ll>>& a) {
8     int n = sz(a); ll ans = 1;
9     rep(i,0,n) {
10        rep(j,i+1,n) {
11            while (a[j][i] != 0) { // gcd step
12                ll t = a[i][i] / a[j][i];
13                if (t) rep(k,i,n)
14                    a[i][k] = (a[i][k] - a[j][k] * t) % mod;
15                swap(a[i], a[j]);
16                ans *= -1;
17            }
18        }
19        ans = ans * a[i][i] % mod;
20        if (!ans) return 0;
21    }
22    return (ans + mod) % mod;
23 }

/* Author: Per Austrin, Simon Lindholm (License: CC0)
2 * Description: Solves $A * x = b$. If there are multiple
   solutions, an arbitrary one is returned.
3 * Returns rank, or -1 if no solutions. Data in $A$ and $b$
   is lost.
4 * Time: $O(n^2 m)$
5 * Status: tested on kattis:equationsolver, and brute-force-
   tested mod 3 and 5 for $n, m \le 3$ */
6 typedef vector<double> vd;
7 const double eps = 1e-12;
8 int solveLinear(vector<vd>& A, vd& b, vd& x) {
9     int n = sz(A), m = sz(x), rank = 0, br, bc;
10    if (n) assert(sz(A[0]) == m);
11    vi col(m); iota(all(col), 0);
12    rep(i,0,n) {
13        double v, bv = 0;
14        rep(r,i,n) rep(c,i,m)
15            if ((v = fabs(A[r][c])) > bv)
16                br = r, bc = c, bv = v;
17        if (bv <= eps) {
18            rep(j,i,n) if (fabs(b[j]) > eps) return -1;
19            break;
20        }
21        swap(A[i], A[br]);
22        swap(b[i], b[br]);
23        swap(col[i], col[bc]);
24        rep(j,0,n) swap(A[j][i], A[j][bc]);
25        bv = 1/A[i][i];
26        rep(j,i+1,n) {
27            double fac = A[j][i] * bv;
28            b[j] -= fac * b[i];
29            rep(k,i+1,m) A[j][k] -= fac * A[i][k];
30        }
31        rank++;
32    }
33    x.assign(m, 0);
34    for (int i = rank; i--;) {
35        b[i] /= A[i][i];
36        x[col[i]] = b[i];
37        rep(j,0,i) b[j] -= A[j][i] * b[i];
38    }
39    return rank; // (multiple solutions if rank < m)
40 }

/* Author: Simon Lindholm (License: CC0)
2 * Description: To get all uniquely determined values of $x$
   back from SolveLinear, make the following changes:
3 * Status: tested on kattis:equationsolverplus, stress-tested
   */
4 #include "SolveLinear.h"

```

```

5 rep(j,0,n) if (j != i) // instead of rep(j,i+1,n)
6 // ... then at the end:
7 x.assign(m, undefined);
8 rep(i,0,rank) {
9   rep(j,rank,m) if (fabs(A[i][j]) > eps) goto fail;
10  x[col[i]] = b[i] / A[i][i];
11 fail:; }

1/* Author: Simon Lindholm (License: CC0)
2* Description: Solves  $Ax = b$  over  $\mathbb{F}_2$ . If there
3are multiple solutions, one is returned arbitrarily.
4* Returns rank, or -1 if no solutions. Destroys  $A$  and  $b$ .
5* Time:  $O(n^2 m)$ 
6* Status: brute-force-tested for  $n, m \leq 4 *$ 
7typedef bitset<1000> bs;
8int solveLinear(vector<bs>& A, vi& b, bs& x, int m) {
9  int n = sz(A), rank = 0, br;
10 assert(m <= sz(x));
11 vi col(m); iota(all(col), 0);
12 rep(i,0,n) {
13   for (br=i; br<n; ++br) if (A[br].any()) break;
14   if (br == n) {
15     rep(j,i,n) if(b[j]) return -1;
16     break;
17   }
18   int bc = (int)A[br]._Find_next(i-1);
19   swap(A[i], A[br]);
20   swap(b[i], b[br]);
21   swap(col[i], col[bc]);
22   rep(j,0,n) if (A[j][i] != A[j][bc]) {
23     A[j].flip(i); A[j].flip(bc);
24   }
25   rep(j,i+1,n) if (A[j][i]) {
26     b[j] ^= b[i];
27     A[j] ^= A[i];
28   }
29   rank++;
30 }
31 x = bs();
32 for (int i = rank; i--;) {
33   if (!b[i]) continue;
34   x[col[i]] = 1;
35   rep(j,0,i) b[j] ^= A[j][i];
36 }
37 return rank; // (multiple solutions if rank < m)
38 }

1/* Author: Max Bennedich
2* Description: Invert matrix  $A$ . Returns rank; result is
3stored in  $A$  unless singular (rank <  $n$ ).
4* Can easily be extended to prime moduli; for prime powers,
5repeatedly
6* set  $A^{-1} = A^{-1} (2I - AA^{-1}) \pmod{p^k}$ 
7where  $A^{-1}$  starts as
8* the inverse of  $A \pmod{p}$ , and  $k$  is doubled in each step.
9* Time:  $O(n^3)$ 
10* Status: Slightly tested */
11int matInv(vector<vector<double>>& A) {
12  int n = sz(A); vi col(n);
13  vector<vector<double>> tmp(n, vector<double>(n));
14  rep(i,0,n) tmp[i][i] = 1, col[i] = i;
15  rep(i,0,n) {
16    int r = i, c = i;
17    rep(j,i,n) rep(k,i,n) if (fabs(A[j][k]) > fabs(A[r][c]))
18      r = j, c = k;
19    if (fabs(A[r][c]) < 1e-12) return i;
20    A[i].swap(A[r]); tmp[i].swap(tmp[r]);
21    rep(j,0,n)
22      swap(A[j][i], A[j][c]), swap(tmp[j][i], tmp[j][c]);
23    swap(col[i], col[c]);
24    double v = A[i][i];
25    rep(j,i+1,n) {
26      double f = A[j][i] / v;
27      A[j][i] = 0;
28      rep(k,i+1,n) A[j][k] -= f*A[i][k];
29      rep(k,0,n) tmp[j][k] -= f*tmp[i][k];
30    }
31    rep(j,i+1,n) A[i][j] /= v;
32    rep(j,0,n) tmp[i][j] /= v;
33    A[i][i] = 1;
34  }
35  /// forget A at this point, just eliminate tmp backward
36  for (int i = n-1; i > 0; --i) rep(j,0,i) {
37    double v = A[j][i];
38    rep(k,0,n) tmp[j][k] -= v*tmp[i][k];
39  }
40  rep(i,0,n) rep(j,0,n) A[col[i]][col[j]] = tmp[i][j];
41  return n;
42 }

```

```

40 }

1/* Author: Simon Lindholm
2* Description: Invert matrix  $A$  modulo a prime.
3* Returns rank; result is stored in  $A$  unless singular
4(rank <  $n$ ).
5* For prime powers, repeatedly set  $A^{-1} = A^{-1} (2I -$ 
6 $AA^{-1}) \pmod{p^k}$  where  $A^{-1}$  starts as
7* the inverse of  $A \pmod{p}$ , and  $k$  is doubled in each step.
8* Time:  $O(n^3)$ 
9* Status: Slightly tested */
10#include "../number-theory/ModPow.h"
11int matInv(vector<vector<ll>>& A) {
12  int n = sz(A); vi col(n);
13  vector<vector<ll>> tmp(n, vector<ll>(n));
14  rep(i,0,n) tmp[i][i] = 1, col[i] = i;
15  rep(i,0,n) {
16    int r = i, c = i;
17    rep(j,i,n) rep(k,i,n) if (A[j][k]) {
18      r = j; c = k; goto found;
19    }
20    found:
21    A[i].swap(A[r]); tmp[i].swap(tmp[r]);
22    rep(j,0,n)
23      swap(A[j][i], A[j][c]), swap(tmp[j][i], tmp[j][c]);
24    swap(col[i], col[c]);
25    ll v = modpow(A[i][i], mod - 2);
26    rep(j,i+1,n) {
27      ll f = A[j][i] * v % mod;
28      A[j][i] = 0;
29      rep(k,i+1,n) A[j][k] = (A[j][k] - f*A[i][k]) % mod;
30      rep(k,0,n) tmp[j][k] = (tmp[j][k] - f*tmp[i][k]) %
31      mod;
32    }
33    rep(j,i+1,n) A[i][j] = A[i][j] * v % mod;
34    rep(j,0,n) tmp[i][j] = tmp[i][j] * v % mod;
35    A[i][i] = 1;
36  }
37  for (int i = n-1; i > 0; --i) rep(j,0,i) {
38    ll v = A[j][i];
39    rep(k,0,n) tmp[j][k] = (tmp[j][k] - v*tmp[i][k]) % mod;
40  }
41  rep(i,0,n) rep(j,0,n)
42    A[col[i]][col[j]] = tmp[i][j] % mod + (tmp[i][j] <
43    0)*mod;
44  return n;
45 }

1/** Author: Ulf Lundstrom, Simon Lindholm (License: CC0)
2* Description: Solves the tridiagonal matrix equation
3system:
4*  $\begin{bmatrix} b_0 & d_0 & p_0 & 0 & 0 & \dots & 0 \\ b_1 & d_1 & p_1 & 0 & 0 & \dots & 0 \\ b_2 & d_2 & p_2 & \dots & 0 & \dots & 0 \\ \dots & \dots & \dots & \dots & \dots & \dots & \dots \\ b_{n-1} & d_{n-1} & p_{n-1} & 0 & q_{n-2} & d_{n-1} \end{bmatrix} \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ \dots \\ x_{n-1} \end{bmatrix}$ 
5*  $\begin{bmatrix} b_2 \\ \dots \\ b_{n-1} \end{bmatrix} = \begin{bmatrix} 0 \\ \dots \\ 0 \end{bmatrix}$ 
6*  $\begin{bmatrix} b_2 \\ \dots \\ b_{n-1} \end{bmatrix} = \begin{bmatrix} 0 \\ \dots \\ 0 \end{bmatrix}$ 
7*  $\begin{bmatrix} b_2 \\ \dots \\ b_{n-1} \end{bmatrix} = \begin{bmatrix} 0 \\ \dots \\ 0 \end{bmatrix}$ 
8* Application:
9* Solves linear recurrence systems of the form:
10*  $a_i = b_i * a_{i-1} + c_i * a_{i+1} + d_i$  ( $1 \leq i \leq n$ )
11* where  $a_0, a_{n+1}, b_i, c_i$ , and  $d_i$  are given.
12* The solution vector  $a$  can be obtained via:
13*  $d = \{ 1, -1, -1, \dots, -1, 1 \}$ 
14*  $p = \{ 0, c_1, c_2, \dots, c_n \}$ 
15*  $q = \{ b_1, b_2, \dots, b_n, 0 \}$ 
16*  $b = \{ a_0, d_1, d_2, \dots, d_n, a_{n+1} \}$ 
17* Numerical Stability:
18* - Fails if the solution is not unique.
19* - Numerically stable without pivot checks if:
20* 1) Strictly diagonally dominant:
21*  $|d_i| > |p_i| + |q_{i-1}|$  for all  $i$ , OR
22*  $|d_i| > |p_{i-1}| + |q_i|$  for all  $i$ 
23* 2) The matrix is symmetric positive-definite.
24* Complexity:  $O(N)$  time,  $O(N)$  space. Status: Brute-force
25tested mod 5 and 7, stress-tested for real matrices. */
26typedef double T;
27vector<T> tridiagonal(vector<T> diag, const vector<T>&
28super,
29const vector<T>& sub, vector<T> b) {
30  int n = sz(b); vi tr(n);
31  rep(i,0,n-1) {
32    if (abs(diag[i]) < 1e-9 * abs(super[i])) { // diag[i] ==
33      0
34      b[i+1] -= b[i] * diag[i+1] / super[i];
35      if (i+2 < n) b[i+2] -= b[i] * sub[i+1] / super[i];
36      diag[i+1] = sub[i]; tr[i+1] = 1;
37    } else {
38      diag[i+1] -= super[i]*sub[i]/diag[i];
39      b[i+1] -= b[i]*sub[i]/diag[i];
40    }
41  }

```

```

38 }
39 for (int i = n; i--;) {
40     if (tr[i]) {
41         swap(b[i], b[i-1]);
42         diag[i-1] = diag[i];
43         b[i] /= super[i-1];
44     } else {
45         b[i] /= diag[i];
46         if (i) b[i-1] -= b[i]*super[i-1];
47     }
48 }
49 return b;
50 }

```

### 13.4 Fourier transforms

```

1/**
2 * Author: chilli, Ludo Pulles, Simon Lindholm (License:
3 * CC0)
4 * Description: Higher precision FFT, can be used for
5 * convolutions modulo arbitrary integers
6 * as long as N\log_2N * mod < 8.6e14 (in practice 10^16 or
7 * higher).
8 * Inputs must be in [0, mod).
9 * Time: O(N \log N), where N = |A| + |B| (twice as slow as
10 * NTT or normal FFT), Status: stress-tested */
11
12typedef complex<double> C;
13typedef vector<ll> vl;
14void fft(vector<C>& a) {
15    int n = sz(a), L = 31 - __builtin_clz(n);
16    static vector<complex<long double>> R(2, 1);
17    static vector<C> rt(2, 1);
18    for (static int k = 2; k < n; k *= 2) {
19        R.resize(n); rt.resize(n);
20        auto x = polar(1.0L, acos(-1.0L) / k);
21        rep(i, k, 2 * k) rt[i] = R[i] = i & 1 ? R[i / 2] * x :
22            R[i / 2];
23    }
24    vi rev(n);
25    rep(i, 0, n) rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
26    rep(i, 0, n) if (i < rev[i]) swap(a[i], a[rev[i]]);
27    for (int k = 1; k < n; k *= 2)
28        for (int i = 0; i < n; i += 2 * k) rep(j, 0, k) {
29            auto x = (double *)&rt[j + k], y = (double *)&a[i + j
30                + k];
31            C z(x[0] * y[0] - x[1] * y[1], x[0] * y[1] + x[1] *
32                y[0]);
33            a[i + j + k] = a[i + j] - z;
34            a[i + j] += z;
35        }
36    }
37    template<int M> vl convMod(const vl &a, const vl &b) {
38        if (a.empty() || b.empty()) return {};
39        vl res(sz(a) + sz(b) - 1);
40        int B = 32 - __builtin_clz(sz(res)), n = 1 << B, cut =
41            int(sqrt(M));
42        vector<C> L(n), R(n), outs(n), outl(n);
43        rep(i, 0, sz(a)) L[i] = C((int)a[i] / cut, (int)a[i] %
44            cut);
45        rep(i, 0, sz(b)) R[i] = C((int)b[i] / cut, (int)b[i] %
46            cut);
47        fft(L), fft(R);
48        rep(i, 0, n) {
49            int j = -i & (n - 1);
50            outl[j] = (L[i] + conj(L[j])) * R[i] / (2.0 * n);
51            outs[j] = (L[i] - conj(L[j])) * R[i] / (2.0 * n) / 1i;
52        }
53        fft(outl), fft(outs);
54        rep(i, 0, sz(res)) {
55            ll av = ll(real(outl[i]) + .5), cv = ll(imag(outs[i])
56                + .5);
57            ll bv = ll(imag(outl[i]) + .5) + ll(real(outs[i]) + .5);
58            res[i] = ((av % M * cut + bv) % M * cut + cv) % M;
59        }
60        return res;
61    }
62}

```

```

1/* Author: Lucian Bicsi (License: GNU Free Documentation
2 * License 1.2)
3 * Description: Transform to a basis with fast convolutions
4 * of the form
5 * c[z] = \sum_{z = x \oplus y} a[x] * b[y]
6 * where \oplus is one of AND, OR, XOR.
7 * The size of $a$ must be a power of 2.
8 * Time: O(N \log N), Status: stress-tested */
9void FST(vi& a, bool inv) {
10    for (int n = sz(a), step = 1; step < n; step *= 2) {
11        for (int i = 0; i < n; i += 2 * step) rep(j, i, i+step) {
12            int &u = a[j], &v = a[j + step]; tie(u, v) =
13                inv ? pii(v - u, u) : pii(v, u + v); // AND

```

```

12 // inv ? pii(v, u - v) : pii(u + v, u); // OR ///
13 include-line
14 // pii(u + v, u - v); // XOR ///
15 include-line
16 }
17 // if (inv) for (int& x : a) x /= sz(a); // XOR only ///
18 include-line
19 }
20 vi conv(vi a, vi b) {
21     FST(a, 0); FST(b, 0);
22     rep(i, 0, sz(a)) a[i] *= b[i];
23     FST(a, 1); return a;
24 }

```

## 14 Combinatorics

### 14.1 Catalan Number

$$C_0 = 1, C_n = \sum_{i=0}^{n-1} C_i C_{n-1-i}, C_n = C_n^{2n} - C_n^{2n-1}$$

|    |        |        |         |         |
|----|--------|--------|---------|---------|
| 0  | 1      | 1      | 2       | 5       |
| 4  | 14     | 42     | 132     | 429     |
| 8  | 1430   | 4862   | 16796   | 58786   |
| 12 | 208012 | 742900 | 2674440 | 9694845 |

### 14.2 Bertrand's Ballot Theorem

- $A$  always  $> B$ :  $C(p+q, p) - 2C(p+q-1, p)$
- $A$  always  $\geq B$ :  $C(p+q, p) \times \frac{p+1-q}{p+1}$

### 14.3 Burnside's Lemma

Let  $X$  be the original set.  
 Let  $G$  be the group of operations acting on  $X$ .  
 Let  $X^g$  be the set of  $x$  not affected by  $g$ .  
 Let  $X/G$  be the set of orbits. Then the following equation holds:

$$|X/G| = \frac{1}{|G|} \sum_{g \in G} |X^g|$$

## 15 Special Numbers

### 15.1 Fibonacci Series

|    |         |         |         |          |
|----|---------|---------|---------|----------|
| 1  | 1       | 1       | 2       | 3        |
| 5  | 5       | 8       | 13      | 21       |
| 9  | 34      | 55      | 89      | 144      |
| 13 | 233     | 377     | 610     | 987      |
| 17 | 1597    | 2584    | 4181    | 6765     |
| 21 | 10946   | 17711   | 28657   | 46368    |
| 25 | 75025   | 121393  | 196418  | 317811   |
| 29 | 514229  | 832040  | 1346269 | 2178309  |
| 33 | 3524578 | 5702887 | 9227465 | 14930352 |

$$f(45) \approx 10^9, f(88) \approx 10^{18}$$

### 15.2 Prime Numbers

- $\pi(n) \equiv$  Number of primes  $\leq n \approx \frac{n}{(\ln n)-1}$
- $\pi(100) = 25, \pi(200) = 46$
- $\pi(500) = 95, \pi(1000) = 168$
- $\pi(2000) = 303, \pi(4000) = 550$
- $\pi(10^4) = 1229, \pi(10^5) = 9592$
- $\pi(10^6) = 78498, \pi(10^7) = 664579$

### 15.3 Number of Divisors

- If  $n = \prod p_i^{a_i}$ , then  $\tau(n) = \prod (a_i + 1)$
- Maximum  $\tau(n)$  for  $n \leq 10^k$ :

| $k$ | $\max \tau(n)$ | $k$ | $\max \tau(n)$ | $k$ | $\max \tau(n)$ |
|-----|----------------|-----|----------------|-----|----------------|
| 3   | 32             | 9   | 1344           | 15  | 26880          |
| 4   | 64             | 10  | 2304           | 16  | 41472          |
| 5   | 128            | 11  | 4032           | 17  | 64512          |
| 6   | 240            | 12  | 6720           | 18  | 103680         |
| 7   | 448            | 13  | 10752          |     |                |
| 8   | 768            | 14  | 17280          |     |                |

- Useful bounds:  $n \leq 10^6 \Rightarrow \tau(n) \leq 240$
- $n \leq 10^9 \Rightarrow \tau(n) \leq 1344$
- $n \leq 10^{12} \Rightarrow \tau(n) \leq 6720$
- $n \leq 10^{15} \Rightarrow \tau(n) \leq 26880$
- $n \leq 10^{18} \Rightarrow \tau(n) \leq 103680$

### 15.4 Distinct Prime Factors

- Let  $\omega(n)$  = number of distinct prime factors.
- Minimum number with  $k$  distinct prime factors is the product of the first  $k$  primes.
  - $2 \cdot 3 \cdot 5 \cdot 7 \cdot 11 \cdot 13 \cdot 17 \cdot 19 = 9699690$



- $2 \cdot 3 \cdot \dots \cdot 23 = 223092870$
- $2 \cdot 3 \cdot \dots \cdot 29 = 6469693230$
- $n \leq 10^9 \Rightarrow \omega(n) \leq 9, n \leq 10^{18} \Rightarrow \omega(n) \leq 15$
- Number of square-free divisors =  $2^{\omega(n)}$ 
  - $n \leq 10^9 \Rightarrow \leq 512, n \leq 10^{18} \Rightarrow \leq 32768$

